Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Робототехника и комплексная автоматизация

КАФЕДРА Компьютерные системы автоматизации производства

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ:

Разработка системы планирования траектории движения группы мобильных роботов в замкнутой среде с препятствиями

Студент РК9-41М О.Е. Ольхов
(группа) (подпись) (инициалы, фамилия)

Руководитель ВКР М.В. Овсянников
(подпись) (инициалы, фамилия)

Нормоконтролер А.Д. Жаргалова
(подпись) (инициалы, фамилия)

2025 год

РЕФЕРАТ

Выпускная квалификационная работа выполнена по теме: «Разработка системы планирования траектории движения группы мобильных роботов в замкнутой среде с препятствиями».

Расчетно-пояснительная записка содержит 60 листов формата А4 и 18 рисунков.

Целью выполнения работы является создание модуля построения траектории для группы мобильных роботов. В работе проведен анализ существующих алгоритмов управления группой мобильных роботов, выявление недостатков и доработка под существующие задачи. В ходе работы описывается разработка алгоритма выбора траектории группой мобильных роботов на заданной карте, тестирование модуля. Заключение посвящено основным выводам по проделанной работе.

Ключевые слова: Python, Группа мобильных роботов.

Объектом автоматизации является гибкая производственная линия.

СОДЕРЖАНИЕ

РЕФЕРАТ 2
СОДЕРЖАНИЕ 3
ВВЕДЕНИЕ 5
1. Анализ объекта автоматизации 8
2 Концептуальное проектирование системы 11
2.1 Анализ двухколесного мобильного робота 11
2.2 Выбор системы управления 14
2.3 Выбор метода определения положения для робота 16
2.4 Выбор технического оснащения для робота 18
2.5 Выбор базового алгоритма построения траектории для робота в неизвестном пространстве 20
2.5.1 Критерии выбора алгоритма 22
2.5.2 Экспериментальная оценка алгоритмов 23
2.5.3 Итоговый выбор алгоритма 25
2.6 Анализ текущего состояния робота, выбор промежуточной позиции 26
2.7 Процедура выбора лидера 28
2.8 Критерии оценки 31
3. Техническое задание 34
3.1 Общие сведения 34
3.2 Назначения и цели создания модуля 34
4. Техническое проектирование и разработка алгоритма 36
4.1 Техническая база 36
4.2 Представление карты складского помещения 37
4.3 Архитектура взаимодействия узлов системы 38
4.4 Алгоритм управления группой роботов 40
5. Рабочее проектирование системы 42
5.1 Создание карты для симуляции 42
5.2 Разработка модуля управления 45
6. Апробирование 50
ЗАКЛЮЧЕНИЕ 55

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 57
ПРИЛОЖЕНИЕ А 61
ПРИЛОЖЕНИЕ Б 70
ПРИЛОЖЕНИЕ В 70

ВВЕДЕНИЕ

Как известно, возможности отдельного мобильного робота, в общем случае, ограниченны, поэтому для решения различных задач, в том числе для перевозки различных грузов, гораздо чаще применяются группы мобильных, безэкипажных роботов [4-6,13]. Такие группы транспортных роботов могут быть однородными или гетерогенными. Одной из существенных особенностей транспортных задач является априорная неопределенность как перевозимых грузов, так и условий транспортировки. В тоже время эти неопределенные факторы имеют важное значение для функционирования системы управления движением мобильных роботов [7, 11]. Транспортируемые грузы могут быть весьма разнообразными как по габаритам, по массе, так и по пути движения.

В настоящее время роботы и робототехнические системы (РТС) получили широкое практическое применение во многих сферах человеческой деятельности. Этот факт обусловлен тем, что данные системы позволяют автоматизировать ряд трудоемких и рутинных задач: мониторинг и ликвидация чрезвычайных ситуаций, спасательные операции, подводные исследования, сельскохозяйственные работы, разведывательные операции, а также многие другие. При этом возникает вопрос эффективности применения РТС в задачах, предполагающих автономное функционирование в течение продолжительного времени (например, исследование больших территорий) или выполнение работ в агрессивных средах (под водой, в космосе, в зонах радиационного загрязнения). В подобных условиях становится актуальным применение нескольких роботов совместно, то есть в группе.

При такой модели управления РТС будет отличаться повышенной надежностью и устойчивостью к внешним возмущениям, а также возможностью масштабирования численности роботов группы в зависимости от сложности решаемой задачи. Для обеспечения указанных преимуществ подобные РТС должны иметь распределённую систему управления для обеспечения высокой степени автономности, что, в свою очередь, приводит к

появлению ряда специфических проблем управления.

Исходя из анализа исследовательских работ [4-6,11,13,19], можно сделать вывод, что одной из ключевых проблем, возникающих при функционировании группы мобильных роботов (МР), является планирование безопасных траекторий движения, позволяющих строить безаварийные маршруты, близкие к оптимальным (кратчайшим).

Цель работы: Разработка системы планирования траектории движения группы мобильных роботов в замкнутой среде с препятствиями

В ходе разработки проекта были поставлены следующие задачи:

1. Сформулировать требования, предъявляемые к разрабатываемой системе.
2. Выбрать оптимальную систему управления роботами.
3. Проанализировать технические характеристики робокара и реализовать систему при минимальных технических доработках.
4. Выбрать оптимальный алгоритм в поставленных условиях.
5. Разработать алгоритмы основных функций программ.
6. Разработать исходный код программной части на языке python на базе открытой робототехнической платформы ros.

Роботизация склада выгодна по многим факторам.

Основные преимущества автоматизированных погрузчиков:

1. Повышение производительности. Робот справляется с любой складской операцией в среднем в 3,5–4 раза быстрее человека.
2. Робот поднимает и переносит тяжелый груз, с которым человек не может справиться без подъемных механизмов.
3. Миниатюрные складские погрузчики не требуют больших проходов, что позволяет экономить полезную площадь.
4. Исключение человеческого фактора. Это приводит к устранению ошибок в сортировке грузов и травмирования людей. Существенно снижается риск случайной порчи товара. Исключение травм у работников сокращает расходы на компенсации причиненного ущерба здоровью.

5. Робот способен работать без отдыха и в длительном режиме.

1. Анализ объекта автоматизации

Исследуемым объектом является ГПС производства фирмы Denford, в частности один из модулей ее транспортно-накопительной системы – группа двухколёсных мобильных роботов. Структурная схема ГПС Denford приведена на рисунке 1.

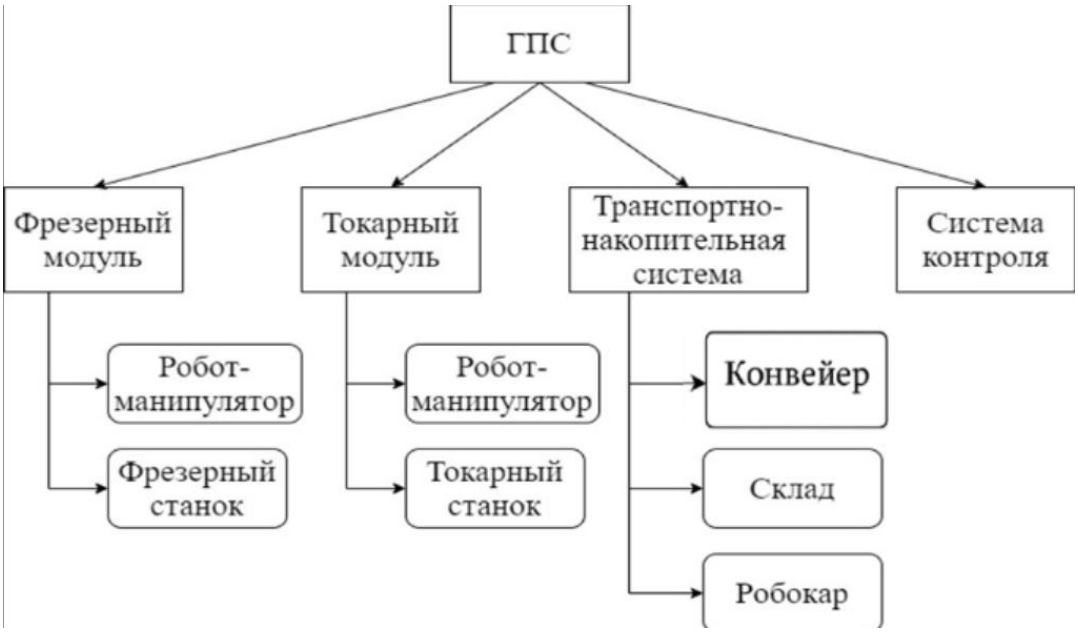


Рисунок 1 – Структурная схема ГПС Denford

Постановка задачи

Двухколёсный мобильный робот с дифференциальным приводом – это автономное транспортное устройство, предназначенное для перемещения грузов или выполнения задач в ограниченном пространстве. Робот оснащён двумя ведущими колёсами и, как правило, одной или двумя опорными точками (например, роликами) для устойчивости. Благодаря дифференциальному приводу управление движением осуществляется путём изменения скорости вращения каждого из ведущих колёс, что позволяет роботу точно маневрировать, разворачиваться на месте и двигаться по заданной траектории. Робот функционирует в автономном режиме без участия оператора, способен работать во вредных или опасных условиях, в местах с высокой концентрацией людей или других мобильных роботов. Он снабжён необходимыми датчиками и системами безопасности, обеспечивающими устойчивую и безопасную

эксплуатацию.

Основные направления работы:

1. Движение по предварительно не известной траектории, включая развилки.

2. Поддержание постоянной скорости в случае сопровождения конвейера или других объектов.

3. Остановки и продолжение движения, как в заранее определенных позициях, так и «по требованию» оператора.

4. Загрузка и разгрузка перевозимых компонентов.

5. Беспроводная связь с другими agv или центральным терминалом для создания сети тележек, движущихся без участия людей.

6. Распознавание перемещающихся препятствий, ожидание их исчезновения и продолжение работы.

Характеристики робокара:

1. Вес тележки – 110 кг;

2. Размеры – 1000x550x375;

3. Минимальный радиус поворота – 1 м;

4. Средняя скорость – 30 м/мин;

5. Среднее время работы – 8 ч;

6. Главный тяговый двигатель – двигатель постоянного тока с постоянным магнитом, приводимый в действие 2-квадрантной системой привода FET;

7. Рулевой двигатель - шаговый двигатель, управляемый шаговым приводом;

8. DENSTEP;

9. Аккумулятор - герметичные свинцово-кислотные батареи;

10. Вход - 250-265 В переменного тока 50/60 Гц;

11. Выход - 25,0-30,0 В 0,8 А;

12. Вес аккумулятора – 0.8 кг;

Исследуемая тележка имеет инфракрасный датчик стыковки и датчик

столкновений.

Основные проблемы существующего решения: Необходимость постоянного присутствия человека при принятии и транспортировке грузов, устаревшее техническое состояние робокара и отсутствие удаленного управления. Данная система не способна автономно перемещать товары и подстраиваться под состояние окружающего пространства.

2 Концептуальное проектирование системы

2.1 Анализ двухколесного мобильного робота

В работе рассматривается двухколесный мобильный робот с дифференциальным приводом, кинематическая схема которого представлена на рисунке 2. На рисунке 2 x и y обозначают положение центра оси робота в глобальной системе координат, θ обозначает направление движения мобильного робота относительно оси x в глобальной системе координат. Переменная r_w есть радиус колес робота, а B – расстояние между колесами. И наконец, v и w являются линейной и угловой скоростью соответственно, а W_L и W_R – скорость вращения левого и правого колес соответственно.

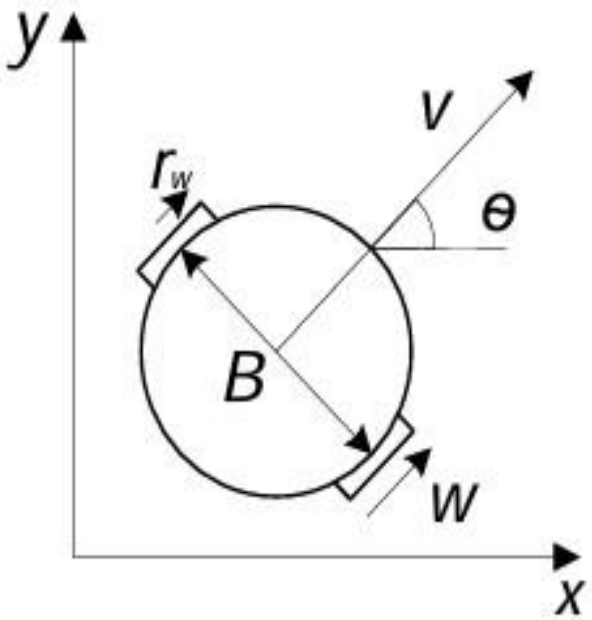


Рисунок 2 – Кинематическая схема мобильного робота

Данная модель в глобальной системе координат является одной из самых простых и минималистических представлений двухколесного мобильного робота. Робот представлен тремя степенями свободы с вектором:

$$q = [x, y, \theta]$$

Математическую модель МР можно представить по формулам 1,2:

$$x^{\cdot} = v cos\theta \quad (1),$$

$$y^{\cdot} = v sin\theta \quad (2).$$

Взаимосвязь между линейной и угловой скоростью и скоростями каждого из колес можно выразить следующим образом по формулам 3,4:

$$v = \frac{r_w}{2} \cdot (W_R + W_L) \quad (3),$$

$$w = \frac{r_w}{2} \cdot (W_R - W_L) \quad (4).$$

Подставив уравнения (1,2) в выражение (3,4), получаем следующую кинематическую модель робота [15]:

$$x^{\cdot} = cos\theta \cdot \frac{r_w}{2} \cdot (W_R + W_L)$$

$$y^{\cdot} = sin\theta \cdot \frac{r_w}{2} \cdot (W_R + W_L)$$

$$\theta^{\cdot} = \frac{r_w}{B} \cdot (W_R - W_L)$$

В данной работе рассматривается группа R, состоящая из N гомогенных мобильных роботов $r_i \in R, i = \{1, \dots, N\}$, при этом:

- Каждый робот $r_i \in R$ оснащен бортовыми устройством управления и датчиками расстояния (лидаром);
- Каждый робот $r_i \in R$ имеет ограниченную область видимости b_i, которую можно представить в виде окружности заданного радиуса r_b с центром, совпадающим с позицией робота. Данные о наличии препятствий $o_i \in O$, получаемые от датчиков расстояния, и определение позиции соседних роботов $r_{i,j} \in R$ осуществляются только в области видимости b_i робота;
- Под неизвестной замкнутой средой будем понимать ситуацию, когда препятствиями также считаются границы рабочей области группы роботов;
- В глобальной системе координат известны положение целевой точки x_g, y_g, а также собственные координаты $x_i(t), y_i(t)$ каждого робота r_i группы в текущий момент времени t.

Процесс планирования движения группы МР предлагается рассмотреть, как задачу пошаговой оптимизации текущего положения робота $x_i(t), y_i(t)$

относительно целевой точки x_g, y_g. [14] Предполагаем, что время t изменяется дискретно и принимает целочисленные значения $0,1,\dots,k$, которые представляют собой интервалы между соседними дискретными моментами времени срабатывания бортовых устройств управления роботов r_i группы.

Состояние каждого робота $r_i \in R$ описывается вектором, компонентами которого являются координаты текущей позиции в глобальной системе координат и угол поворота по формуле 5:

$$s_i(t) = x_i(t), y_i(t), \theta_i(t) \quad (5).$$

В каждый момент времени t на каждого из роботов оказывается управляющее воздействие посредством вектора управления по формуле 6, компонентами которого являются линейная и угловая скорости каждого робота $r_i \in R$:

$$u_i(t) = v_i(t), w_i(t) \quad (6).$$

Таким образом, в каждый момент времени t состояние системы характеризуется вектором $s_i(t)$, а управляющее воздействие – вектором $u_i(t)$. Под влиянием выбранного в момент t управления (принятого решения) система переходит в следующий момент времени в новое состояние. Зависимость между выражениями (5) и (6) можно описать соотношением 7:

$$s_i(t) = f\big(s_i(t-1), u_i(t)\big), t = 0,1,\dots,k \quad (7).$$

Расстояние, которое характеризуется разницей между координатами робота на текущем и предыдущем шагах t и $t-1$ соответственно будем называть сегментом траектории. Тогда общую длину траектории, пройденную группой роботов при движении к цели, можно представить как сумму сегментов траектории, которые необходимо пройти на каждом шаге по уравнению 8:

$$Y = f_i + f_{i+i} + \dots + f_k \quad (8).$$

С целью предотвращения коллизий между МР и столкновений с препятствиями вводятся ограничения на позицию роботов по формуле 8 и неравенству 9:

$$\{d \geq |s^{i-sj \geq \Delta_d,(i \neq j;i,j=1,N)} \quad (8),$$

$$|s_i - o_i| \geq \Delta_o \quad (9),$$

где Δ_d– минимально допустимая дистанция между соседними роботами, Δ_o– минимально допустимая дистанция между роботом $r_i \in R$ и препятствием, находящемся в области видимости b_i робота.

В качестве показателя эффективности примем длину траектории, необходимую для перемещения всех роботов группы из начального положения в целевое. Подставив выражение (7) в (8), получим аддитивную целевую функцию 10:

$$Y = \sum^{k} t = 1\, f\big(s_i(t-1), u_i(t)\big) \quad (10).$$

Таким образом, задачу пошаговой оптимизации процесса планирования траектории движения группы мобильных роботов в неизвестной замкнутой среде с препятствиями можно сформулировать так: определить для каждого робота группы $r_i \in R$ набор управлений $u_i(t), u_i(2), \dots, u_i(k)$, переводящих систему из начального состояния $s_i(0)$ $i = \{1, \dots, N\}$ в конечное состояние $s_i(k)$, $i = \{1, \dots, N\}$ с учетом ограничений (9) так, чтобы целевая функция (10) достигла минимума:

$$Y \to \min$$

2.2 Выбор системы управления

Системы группового управления мобильными роботами являются одним из ключевых элементов, определяющих эффективность и надежность работы коллективов подвижных агентов в различных областях, от промышленной автоматизации до логистических решений и сервисных роботов. Все системы группового управления делятся на 3 основных типа:

1. Централизованные системы – системы, состоящие из группы мобильных роботов и центра управления, который распределяет цели между мобильными роботами, а также проводит локальное планирование для каждого

из них. Таким образом в данной системе отсутствует необходимость установки мощных вычислительных средств для каждого робота, но нужен один компьютер, способный проводить все необходимые операции для каждого робота за короткий промежуток времени. Одним из важных преимуществ централизованного управления является легкость внедрения и отладки — все процессы можно контролировать и корректировать из единой точки, что особенно актуально в условиях ограниченного бюджета и небольшой размерности робототехнического парка. Вместе с тем рост числа мобильных агентов вызывает экспоненциальное увеличение вычислительной нагрузки на центр управления и предъявляет высокие требования к пропускной способности канала связи; в случае отказа центрального узла работа всей системы блокируется.

2. Децентрализованные системы – системы, состоящие из группы мобильных роботов, каждый из которых имеет на борту достаточную вычислительную мощность для реализации системы локального планирования [12, 16-18]. Данные системы являются более надежными, поскольку поломка одного робота не приведет к сбою всей системы, так как задачи этого робота могут быть быстро перераспределены среди остальных участников этой группы. Кроме того, при добавлении нового робота в группу, сложность задачи возрастает линейно, что делает данную системы более эффективной для большого числа роботов. Однако, отсюда следует один из главных недостатков этой системы: высокая стоимость каждого конкретного робота при большом их количестве.

3. Гибридные (смешанные) – системы, состоящие из подгрупп роботов, к каждой из которых привязан свой центр управления. Такой подход позволяет при сохранении гибкости децентрализованного управления централизованно балансировать глобальные цели и обеспечивать взаимосвязь между подгруппами [20]. Гибридные системы демонстрируют более высокую отказоустойчивость по сравнению с чисто централизованными благодаря распределению нагрузки, однако требуют разработки многоуровневых

протоколов и дополнительных ресурсов для обслуживающих локальных серверов. Эта система используется при большом наборе роботов в группе и совмещает в себе преимущества и недостатки централизованных и децентрализованных систем.

В данной работе была рассмотрена централизованная система управления поскольку такой подход требует наличия только одного высокопроизводительного вычислительного центра, что значительно сокращает затраты на оснащение каждого отдельного робота сложной и дорогостоящей аппаратурой. Это особенно актуально в условиях ограниченного бюджета. Кроме того, централизованная система отличается простотой реализации и настройки, так как все процессы управления, отладки и выявления ошибок выполняются из единого центра. Это сокращает время на внедрение и существенно уменьшает вероятность технических сбоев.

Централизованный подход также является наиболее подходящим при небольшом количестве мобильных роботов, поскольку в таких условиях он демонстрирует наибольшую эффективность и производительность. В рамках текущей задачи количество роботов не велико, поэтому нет необходимости в использовании более сложных и затратных децентрализованных решений. Важной особенностью централизованной системы является высокая степень контроля и координации действий всех роботов, что позволяет обеспечить точность и согласованность их работы.

2.3 Выбор метода определения положения для робота

Современные автономные складские робокары имеют множество различных решений по транспортировке грузов. Одной из наиболее острых проблем, стоящих перед современными роботами, является навигация в больших помещениях. Существующие методы определения положения можно разделить на глобальные и локальные. Глобальные методы используют для определения текущего местоположения на обширном пространстве. Локальными способами определения текущего местоположения являются

такие, которые позволяют очень точно определить текущее местоположение в относительно небольшом пространстве [3].

Глобальные методы:

- *Gps*. *Gps* не работает в помещении, когда не видны спутники. Нет возможности определения этажа в многоэтажном здании. – оптические системы определение текущего местоположения по ориентирам, выявленными при помощи машинного зрения является наиболее перспективным и сложным способом;
- *Bluetooth/wifi* beacons или данные о сигналах сбор данных о трех и более ближайших *bluetooth/wifi* точках позволяет рассчитать примерное местонахождение относительно этих точек.

Локальные методы:

- Лидар лазерный радар, постоянно считывающий расстояния до окружающих препятствий при помощи лазерного дальномера. Довольно хорошая точность. Не требуется установка дополнительных маяков;
- Одометрия — использование данных о движении приводов для оценки перемещения. Очень хорошая точность на малых расстояниях, пока нет препятствий и пока имеется хорошее сцепление колес с полом. Без корректировки ошибка накапливается очень быстро и делает вычисление положения невозможным.

В условиях существующей системы небольшого замкнутого складского помещения локальные методы картографирования позволят точнее определить местоположение робокара. В качестве оптимального решения задачи выбран лидар. Лидар – технология получения и обработки информации об удалённых объектах с помощью активных оптических систем, использующих явления отражения света и его рассеивания в прозрачных и полупрозрачных средах.

Лидар как прибор представляет собой, как минимум, активный дальномер оптического диапазона. Сканирующие лидары в системах машинного зрения формируют двумерную или трёхмерную картину окружающего пространства.

Принцип действия лидара не имеет больших отличий от радара: направленный луч источника излучения отражается от целей, возвращается к источнику и улавливается высокочувствительным приёмником (в случае лидара – светочувствительным полупроводниковым прибором); время отклика прямо пропорционально расстоянию до цели.

В отличие от радиоволн, эффективно отражающихся только от достаточно крупных металлических целей, световые волны подвержены рассеиванию в любых средах, в том числе в воздухе, поэтому возможно не только определять расстояние до непрозрачных (отражающих свет) дискретных целей, но и фиксировать интенсивность рассеивания света в прозрачных средах. Возвращающийся отражённый сигнал проходит через ту же рассеивающую среду, что и луч от источника, подвергается вторичному рассеиванию, поэтому восстановление действительных параметров распределённой оптической среды – достаточно сложная задача, решаемая как аналитическими, так и эвристическими методами.

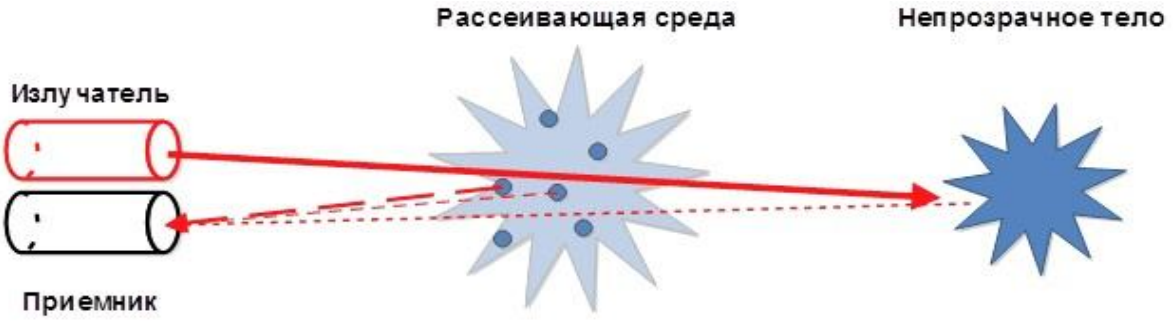


Рисунок 3 – Принцип действия лидара

2.4 Выбор технического оснащения для робота

Алгоритм было принято реализовать на базе открытой робототехнической платформы *ros*. *Ros* обеспечивает стандартные службы операционной системы, такие как: аппаратную абстракцию, низкоуровневый контроль устройств, реализацию часто используемых функций, передачу сообщений между процессами, и управление пакетами. *Ros* основан на архитектуре графов, где обработка данных происходит в узлах,

которые могут получать и передавать сообщения между собой, и работает по принципу «издатель/подписчик».

Библиотека ориентирована на *unix*-подобные системы. Механизм «издатель/подписчик» основан на том, что каждая нода (процесс) регистрируется в центральном «мастере» и через него узнаёт, как связаться с другими узлами. При запуске узел объявляет, какие топики (темы) она будет публиковать и какие топики она будет слушать. Мастер ведёт «таблицу соответствия»: кто издатель какого топика, а кто подписчик, и пересылает нужным нодам *IP*-адрес и порт для установления прямого соединения по протоколу *TCPROS* или *UDPROS*.

В качестве контроллера для узла был рассмотрен выбор между микроконтроллером *Arduino* и одноплатным компьютером *raspberry pi*. При использовании *arduino* для робототехнических проектов существует одно существенное ограничение: *arduino* это микроконтроллер с очень ограниченными вычислительными ресурсами и без полноценной операционной системы. По этой причине на нём невозможно запустить полноценный узел *ROS*. Для корректного функционирования узла системы контроллер должен обладать следующими свойствами.

- Операционную систему – *Arduino* не имеет операционной системы, это микроконтроллер с простым загрузчиком.
- Достаточно оперативной памяти и хранилища – На *Arduino Uno* всего 2 КБ оперативной памяти и 32 КБ флеш-памяти, что недостаточно для большинства *ROS*-компонентов.

Наилучшим решением будет использование одноплатного компьютера *raspberry pi*. Его выбирают в качестве вычислительной платформы потому, что у него достаточно мощный процессор и достаточно оперативной памяти для поставленных задач. *Raspberry Pi* работает под управлением полноценной *Linux* - системы, где есть удобная система пакетов и доступ к репозиториям с библиотеками. Существующие библиотеки на *ros* для работы с лидаром, такие как */rplidar* позволяют максимально эффективно и быстро реализовывать

задуманные решения.

Таким образом, на основании анализа требований к вычислительной мощности, объёму памяти, наличию операционной системы и удобству разработки, а также с учётом возможности быстрого развертывания и последующего расширения функционала, был выбран одноплатный компьютер *Raspberry Pi* в сочетании с лидаром. Такое решение обеспечивает баланс между ценой, производительностью и гибкостью, позволяя реализовать все элементы системы планирования траекторий в рамках *ros* без необходимости значительных переделок аппаратной части.

2.5 Выбор базового алгоритма построения траектории для робота в неизвестном пространстве

При планировании движения мобильных роботов в замкнутой среде с препятствиями были рассмотрены разные алгоритмы построения траектории.

Существует несколько основных классов алгоритмов планирования траектории для мобильных роботов:

- Алгоритмы поиска на графах

Алгоритмы поиска на графах представляют собой класс методов, в которых пространство представляется в виде графа или сетки. Наиболее распространённый представитель этого класса — алгоритм *A** и его модификации. Эти алгоритмы обеспечивают поиск кратчайших или оптимальных по заданному критерию путей на заранее известной или динамически обновляемой карте. Отличаются высокой точностью и возможностью учёта различных факторов среды, но требуют значительных вычислительных ресурсов, особенно в условиях частых изменений в окружении.

Исторически алгоритмы поиска в ширину и поиска в глубину редко применяются в чистом виде для сложных задач, так как обладают низкой эффективностью при наличии большого числа препятствий. Для склада, где

карта заранее неизвестна, невозможно использовать классический алгоритм *A**. Он предполагает, что вся сетка уже построена, и при обнаружении нового препятствия вынужден пересчитывать маршрут целиком. Гораздо практичнее использовать семейство *D** — динамические алгоритмы, которые хранят результаты предыдущего поиска и при появлении новой информации, обновляют только изменённые участки графа.

- Алгоритмы на основе потенциальных полей

Алгоритмы на основе потенциальных полей опираются на физические аналогии силового поля, где препятствия и цели формируют области отталкивания и притяжения, соответственно. Метод искусственных потенциалов отличается простотой реализации и оперативностью реакции на появление препятствий. Основной недостаток данного метода заключается в риске возникновения локальных минимумов, когда робот может застрять в неоптимальной позиции, не достигая цели.

- Жадные алгоритмы

Жадные алгоритмы основываются на локальном выборе точки перемещения робота на каждом шаге без предварительного построения полного маршрута. На каждом шаге робот выбирает движение, максимально приближающее его к текущей цели, при этом обходя обнаруженные препятствия.

Подобный подход позволяет существенно упростить общую архитектуру системы планирования: убираются сложные модули глобального поиска, уменьшается количество вычислений и минимизируется объём хранимых данных о карте местности. В результате внедрение жадных стратегий нередко может быть реализовано даже на бюджетных контроллерах с ограниченным объёмом оперативной памяти и невысокой тактовой частотой процессора. Несмотря на то, что жадный алгоритм не гарантирует глобальной оптимальности траектории, он способен эффективно и оперативно

формировать рабочие траектории, избегая препятствия и последовательно приближаясь к целевым точкам.

2.5.1 Критерии выбора алгоритма

В качестве основных критериев по оценке алгоритма были приняты следующие параметры:

- Время расчёта;
- Длина пути;
- Плавность;
- Масштабируемость для N роботов.

Быстрое время расчета алгоритма критически важно при работе на складских в складских помещениях. При постоянном перемещении объектов внутри помещения существует острая необходимость своевременно реагировать на появляющиеся препятствия. При медленном планировании возможно возникновение ситуаций, в которых будут происходить долгие остановки или ситуация, когда происходит бесконечный пересчет траектории для отдельного робота.

Длина пути напрямую отражается на производительности склада. Чем короче траектория, тем быстрее товар доедет до точки выдачи и тем меньше километраж и износ колёс, редукторов и аккумуляторов.

Плавность траектории – это характеристика оценки углов и резкости поворотов. При рассмотрении двухколесной платформы с дифференциальным приводом резкая смена траектории является причиной износа приводов робота. Так же этот критерий немаловажен из-за того, что резкая смена движения небезопасны при перевозке крупногабаритных грузов и могут привести к опрокидыванию.

Масштабируемость определяет, насколько дееспособен алгоритм при расширении размеров склада. В идеальном случае сложность возрастает линейно и позволяет без лишних усилий увеличивать нагрузку на склад.

Для каждой метрики был введен весовой коэффициент $w_i \in [1,5]$,

который отражает приоритет оцениваемого критерия. В итоговой шкале весов наибольший приоритет (5) отдан времени расчёта. Длина пути и плавность имеют значение 4. Минимальный вес (2) присвоен масштабируемости по числу роботов, потому в данной задаче мы рассматриваем небольшое количество роботов.

2.5.2 Экспериментальная оценка алгоритмов

В процессе выбора алгоритма была проведена экспериментальная оценка трех выбранных методов. Для комплексного тестирования было создано три различных варианта карты с разными начальными условиями, имитирующими реальные складские помещения.

Карты, использованная для тестирования работы алгоритма, была выбрана исходя из нескольких важных критериев. Максимальная приближенность к реальному складскому помещению. Представляет собой замкнутую среду, содержащую узкие проходы и препятствия, характерные для типичного склада. Местность содержит повторяющиеся элементы, которые специально подобраны для повышения сложности задачи локализации роботов и проверки точности работы алгоритма избегания препятствий и координации движения нескольких роботов.

По итогам проведенных экспериментов были получены результаты, которые графически отображены на рисунках 4-6. При проведении тестов на заданных картах потенциальный алгоритм не достигает конечной цели и поэтому его общая длина пути составляет <5 м. В связи с этим мы не можем рассматривать данный метод как возможный способ решения нашей задачи.

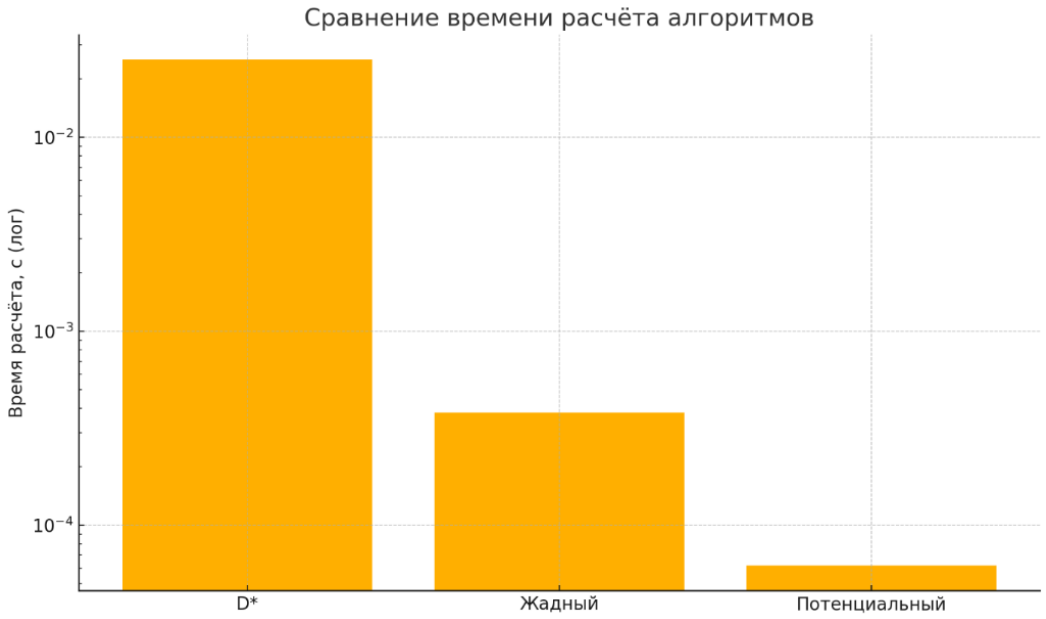


Рисунок 4 – Сравнение среднего времени планирования траектории

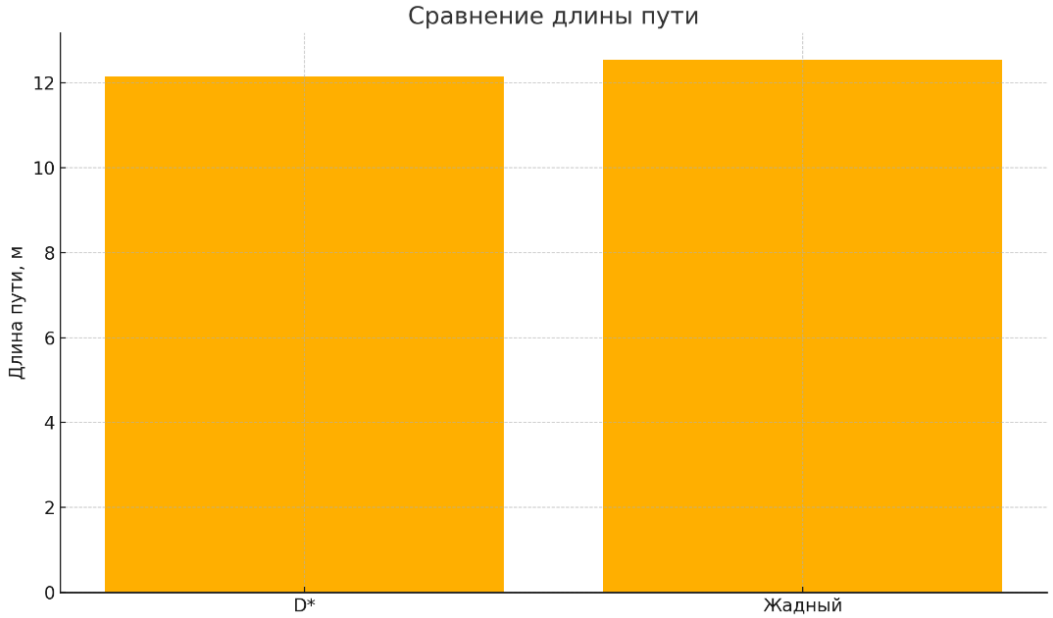


Рисунок 5 – Сравнение средней длины пути

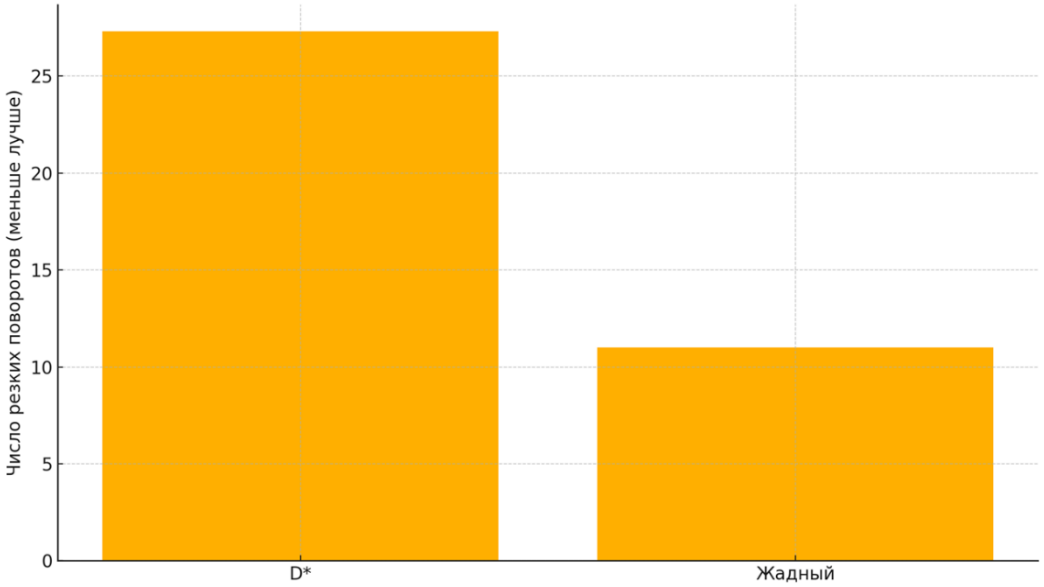


Рисунок 6 – Сравнение плавности траектории

Для получения итоговой оценки по каждому из алгоритмов произвелась нормализация по формуле (11):

$$s_i = \frac{\min}{x_i} \quad (11),$$

где min – минимальное значение среди всех экспериментов,

x_i – результат значения для i-го алгоритма.

По интегральной шкале 0-15 с учётом заданных весовых коэффициентов лидирует жадный алгоритм: его итоговый балл ≈ 10,67. На втором месте оказался D^* с результатом ≈ 5,41.

2.5.3 Итоговый выбор алгоритма

В ходе анализа существующих методов [1,2,10] в качестве базового для данной работы был выбран жадный алгоритм. Основная причина – его простота реализации и оперативная реакция на локальные изменения окружения, что особенно важно в частично неизвестной или динамически меняющейся среде. При этом жадный алгоритм не гарантирует глобально оптимальный путь, однако он позволяет быстро генерировать рабочую траекторию, избегая препятствий и ориентируясь на ближайшую цель или промежуточную точку.

Таким образом, жадный метод служит удобной основой, на которой впоследствии реализуется и тестируется дополнительный функционал

планирования в замкнутой среде с препятствиями.

2.6 Анализ текущего состояния робота, выбор промежуточной позиции

Процесс планирования траектории движения группы мобильных роботов предлагается декомпозировать на две процедуры. Первая процедура – выбор управления – предполагает обработку результатов измерений датчиков расстояния, на основе которых рассчитывается промежуточная позиция в направлении цели так, чтобы обеспечить безаварийное движение робота.

Суть процедуры выбора управления состоит в следующем. В начале считываются измерения датчиков расстояния, которые можно записать как вектор по формуле 12:

$$L(t) = [l_1, l_2, l_3] \quad (12),$$

где m – количество датчиков расстояний, установленных на корпусе каждого из роботов под углом α друг от друга. Предполагается, что один из датчиков l_m измеряет расстояние до препятствия в направлении, совпадающим с ориентацией робота θ_i (рисунок 7).

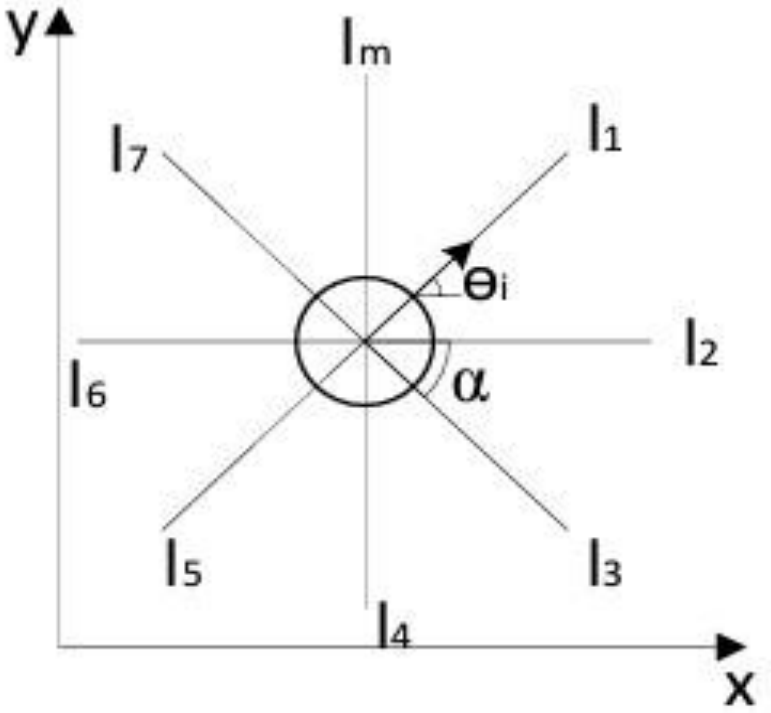


Рисунок 7 – Иллюстрация расположения датчиков расстояния

Углы θ_m, определяющие направление измерений датчиков расстояний, могут быть рассчитаны следующим образом:

$$\theta_m = \frac{360}{ma}$$

Далее необходимо рассчитать координаты точек $p_m = (x_m, y_m)$ в каждом из направлений θ_m, определяющих новый сегмент траектории движения робота r_i.

Координаты точек x_m, y_m рассчитываются на заданном расстоянии h от текущей позиции робота r_i по формулам 13,14:

$$x_m = x_i(t) + hcos\theta_m \quad (13),$$

$$y_m = y_i(t) + hsin\theta_m \quad (14).$$

Предполагается, что величина h задается изначально, при этом ее значение не должно превышать безопасную дистанцию до соседних роботов Δ_d, так как на шаге t+1 их траектории могут пересекать друг друга, а также не должно превышать безопасную дистанцию до препятствий Δo, что указано в выражении 15:

$$\begin{cases} h \leq \Delta d \\ h \leq \Delta o \end{cases} \quad (15).$$

После этого необходимо найти наименьшее расстояние от каждой из точек pm до цели x_g, y_g и выбрать соответствующую точку в качестве нового сегмента траектории движения p_i робота r_i по формулам 16 и 17:

$$d_m = \sqrt{(x_g - x_m)^2 + (y_g - y_m)^2} \quad (16),$$

$$\forall d_m: (d_m \leq d_{m-1} \rightarrow p_i = p_m) \quad (17),$$

где d_m – расстояние от точки pm до цели x_g, y_g.

Тогда можно найти вектор движения для робота по формуле 18:

$$u_i(t) = p_m p_i \quad (18),$$

где $p_m = (x_m, y_m)$ – точка нового сигмента траектории,

$p_i = (x_i, y_i)$ – точка текущего положения робота.

Таким образом, блок-схема описанной выше процедуры для выбора управления представлена на рисунке 8

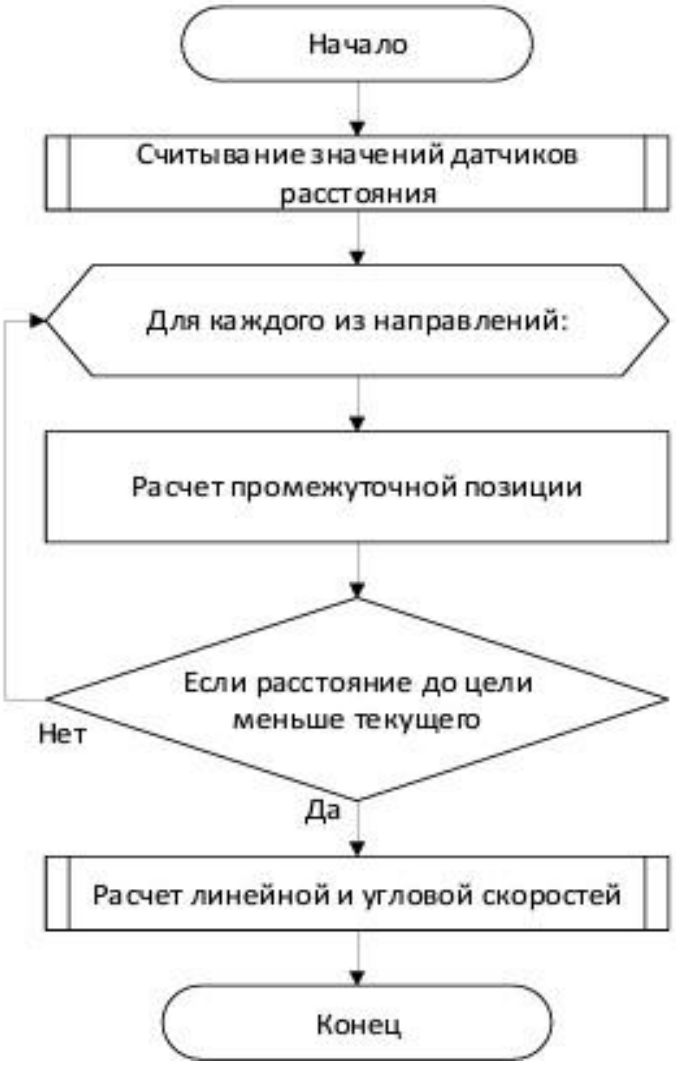


Рисунок 8 – Блок-схема процедуры выбора управления

2.7 Процедура выбора лидера

Применение процедуры выбора управления может быть достаточно для эффективного планирования траектории движения в неизвестной замкнутой среде для одного мобильного робота. В случае группы роботов, где $N > 2$ могут возникнуть ситуации, когда ни одна из возможных позиций при расчете нового сегмента траектории не соответствует ограничению (8 и 9). Так, на рисунке 9 представлен пример проблемной ситуации, когда левый нижний робот оказался заблокирован между препятствиями и другими роботами группы: синие линии показывают возможные направления движения в соответствии с измерениями датчиков расстояния, а красные линии – направления движения, которые заблокированы препятствиями или другими

агентами.

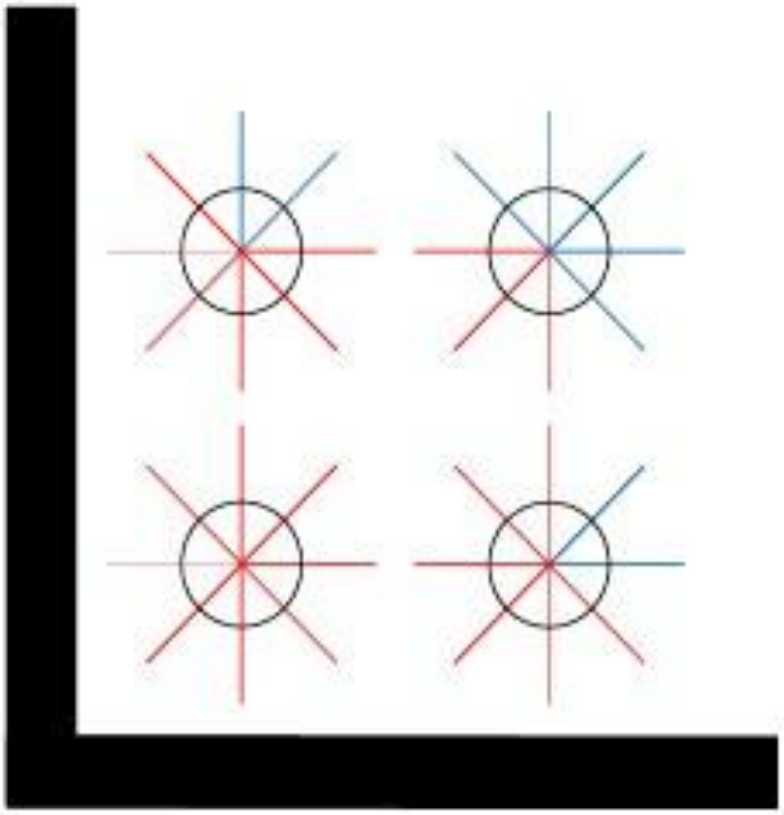


Рисунок 9 – Иллюстрация проблемной ситуации

Для того, чтобы решить подобную проблемную ситуацию в рамках данной методики предлагается вторая процедура – выбор лидера, предназначенная для динамического определения индивидуального лидера для каждого из роботов $r_i \in R$. Процедура состоит из следующих шагов. Вначале определяется множество позиций соседних роботов $r_{i,j}$ группы, входящих в область видимости $b_i, i \neq j;\ i, j = (1,2, \dots, N)$, путем расчета расстояния до каждого из соседних роботов согласно формуле (16). Далее в качестве индивидуального лидера выбирается тот робот, позиция которого:

- Удовлетворяет ограничению (8 и 9);
- Имеет наименьшее расстояние до текущего робота Δ_d рассчитывается аналогично выражению (17).

Таким образом, позиция выбранного лидера считается новым сегментом траектории и на ее основе рассчитается вектор для робота r_i по формуле (18). Данная процедура повторяется до выхода из проблемной ситуации, т.е. до того

момента, когда появится возможность запустить процедуру выбора управления на основе измерений бортовых датчиков расстояния.

Однако, возможна ситуация, когда несколько роботов выберут одного и того же лидера на шаге t. Для избежания столкновения в подобной ситуации предлагается использовать процедуру выбора управления, описанную ранее, с целью корректировки позиции относительно индивидуального лидера, соседних роботов группы и препятствий. Блок-схема процедуры выбора лидера представлена на рисунке 10.

Рисунок 10 – Блок-схема процедуры выбора лидера

Таким образом, блок-схема предложенной методики (рисунок 11) состоит из условно последовательного использования процедур выбора лидера и управления. Критерием останова является достижение цели группой роботов.

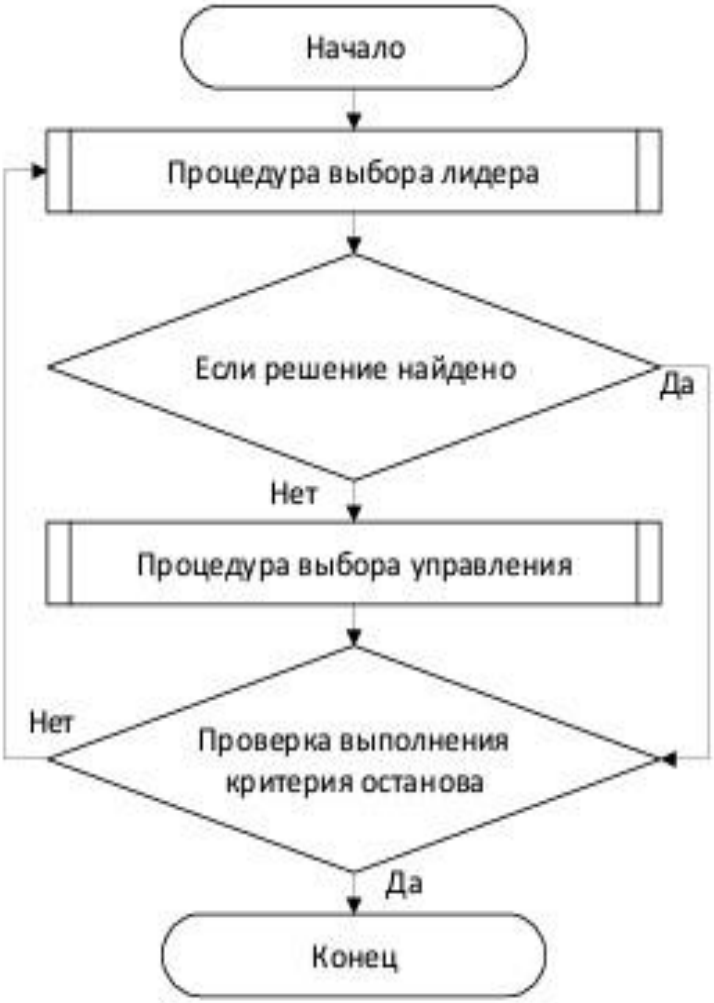


Рисунок 11 – Блок-схема методики планирования траектории движения группы мобильных роботов в замкнутой среде с препятствиями

2.8 Критерии оценки

Наиболее объективными критериями оценки траектории движения робота являются длина и плавность траектории, а также ее безопасность – расстояние до препятствий.

Длина траектории представляет собой общее расстояние, пройденное роботом от начальной точки до целевой. Математически этот критерий заключается в нахождении расстояния между двумя точками, которое

рассчитывается с использованием формулы (16). Этот критерий напрямую влияет на время выполнения задачи и расход энергии, поскольку каждый дополнительный метр пути требует дополнительных усилий приводов. С практической точки зрения сокращение длины траектории позволяет снизить износ колес и узлов подвески, уменьшить вибрационные нагрузки на сенсоры и электронику

Плавность траектории характеризует количество и крутизну поворотов, которые выполняет робот в процессе движения. Чем реже и мягче приходится менять направление, тем более устойчивы параметры движения: снижается нагрузка на сервоприводы, уменьшается риск заноса или резкого срыва в скольжение, упрощается удержание заданной скорости и повышается точность позиционирования. Плавность можно определить как среднее значение углов поворота, которые необходимо сделать роботу при перемещении от начальной точки к конечной.

Математически этот критерий заключается в определении угла между двумя линиями по формуле, показанной в уравнении (19). Кроме того, перед нахождением угла между любыми двумя линиями, углы поворота вычисляются по формуле, показанной в уравнении (20). Наконец, среднее значение угла поворота для оценки гладкости траектории можно найти по формуле (21). Меньший средний угол означает, что траектория довольно гладкая, чтобы робот мог перемещаться от начальной точки к конечной, не делая слишком много поворотов, особенно крутых.

$$tan\theta' = |\frac{p'(t) - p'(t-1)}{1 + p'(t) \cdot p'(t-1)}| \quad (19),$$

$$p' = \frac{y_j(t) - y_j(t-1)}{x_j(t) - x_j(t-1)} \quad (20),$$

$$\bar{\theta} = arctg\left(\frac{1}{k}\sum_{t=1}^{k} sin\theta'(t), \frac{1}{k}\sum_{t=1}^{k} cos\theta'(t),\right) \quad (21).$$

Оценка безопасности траектории заключается в вычислении кратчайшего расстояния от робота до препятствий, а также до других роботов. Кратчайшая

дистанция до препятствий может быть получена на основе измерений датчиков расстояния согласно выражению (22 и 23):

$$\{lmin = \arg min, \quad (22),$$

$$lobs \geq lmin \rightarrow lobs = lmin\,. \quad (23).$$

Аналогично в рамках процедуры выбора управления может быть получено значение минимальной дистанции до соседних роботов группы $r_{i,j}$ по формулам (16) и (17).

3. Техническое задание

3.1 Общие сведения

Полное наименование модуля и его условное обозначение

- Алгоритм управления группы мобильных роботов

Наименование предприятий разработчика и заказчика модуля и их реквизиты

- Разработчик: Ольхов Е. О., РК9-41М.
- Заказчик: Кафедра РК9 МГТУ им. Н. Э. Баумана.
- Юридический адрес: г. Москва, 2-я Бауманская улица, 5 стр.1.

Плановые сроки начала и окончания работы по созданию системы

- Начало работы:
- Окончание работы:

3.2 Назначения и цели создания модуля

Назначение разрабатываемой системы – обеспечение автономности транспортировки грузов на складском помещении с помощью группы мобильных роботов

Модуль обеспечивает:

- Картографирование местности и определение окружающих обьектов;
- Локализация и точное определение нахождения группы роботов.

Требования к реализации модуля:

Разработанный алгоритм должен соответствовать следующим критериям:

- Стабильно распознавать объектов внутри складского помещения;
- Построить оптимальный маршрут перемещения для каждого робота группы;
- Разработанный исходный код программной части должен быть написан на языке *Python*;

- Обеспечить безопасность перемещения и не допускать столкновения роботов группы.

Поставлены следующие задачи:

- Составить техническое задание для разрабатываемой системы
- Проанализировать объект и требования
- Выбрать архитектуру управления и навигации
- Сравнить алгоритмы планирования
- Разработать алгоритмы основных функций программ
- Разработать исходный код программной части на языке python на базе открытой робототехнической платформы ros

4. Техническое проектирование и разработка алгоритма

4.1 Техническая база

Для проведения испытаний и проверки работы системы использовался робот с открытым персональным кодом turtlebot (рисунок 12). Данное устройство имеет всю необходимую техническую базу для реализации системы ориентации в пространстве. Главным преимуществом TurtleBot3 является его модульная конструкция и гибкий выбор конфигурации: пользователь может легко менять шасси, дополнять платформу камерой, дополнительными датчиками или вычислительными ускорителями, не внося при этом серьезных изменений в программную составляющую.

В данной работе выбрана конфигурация «Burger» благодаря её отличной совместимости с дистрибутивом ROS Melodic (Ubuntu 18.04), на базе которого разрабатывался основной код. При организации испытаний была использована среда Gazebo — один из самых распространённых симуляторов роботов с открытым исходным кодом. Для интеграции модели TurtleBot3 в виртуальное пространство применялись ROS-пакеты turtlebot3_gazebo, обеспечивающие корректную загрузку физических характеристик шасси, параметры динамики и имитацию одометрических, оптических и лазерных данных. Это позволило воссоздать реалистичную среду с различными типами препятствий и оценить эффективность алгоритма планирования на этапе симуляции.

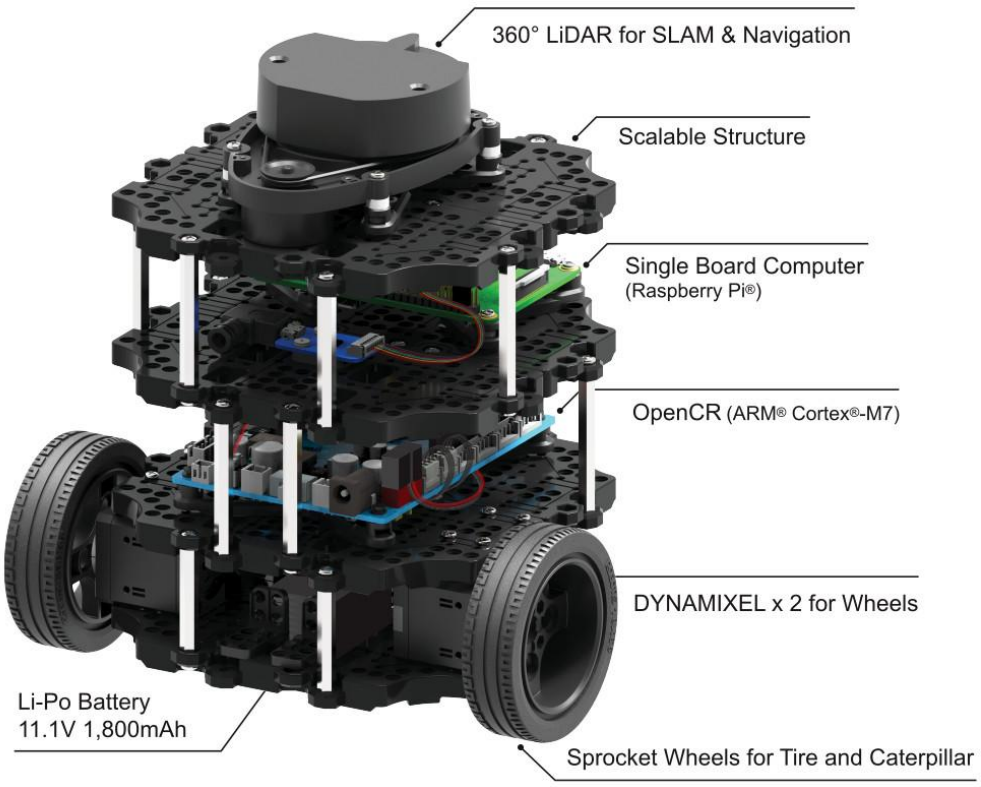


Рисунок 12 – Робот

4.2 Представление карты складского помещения

В большинстве алгоритмов построения траектории данные об окружающем пространстве хранится в так называемой карте-сетке (grid map), которая представляет собой матрицу, каждый элемент которой описывает прямоугольный участок пространства $m_{i,j}$, где i – изменяется от 0 до n_y и j – от 0 до n_x, в самой простой реализации карты-сетки, если элемент матрицы равен 1, то это означает, что в соответствующем участке находится объект, если элемент равен 0, то участок свободен. Данное представление сравнимо с бинарным растровым изображением.

При инициализации модели рабочее пространство сначала задаётся начальными параметрами: реальными размерами склада или цеха, положением и габаритами препятствий. В результате формируется изначальная матрица, в которой каждая ячейка задана с учётом всех начальных данных.

По мере работы мобильного робота, карта-сетка постоянно обновляется: данные с датчиков обновляют существующую карту, корректируя или уточняя

положение объектов и добавляя новые — например, случайно упавший ящик или человека, вошедшего в рабочую зону. Такие правки карты позволяют алгоритмам планирования маршрута учитывать не только статические преграды, но и непрерывно меняющееся окружение, поддерживая безопасность и предотвращая столкновения. Данные используются для контроля столкновений и обеспечения безопасных дистанций.

Таким образом с помощью использования карты-сетки возможно обеспечить безопасность и надёжность работы группы мобильных роботов. Лёгкость реализации, простота хранения и способность к динамическому обновлению делают такое представление территории идеальным выбором для систем навигации и планирования траектории в реальном времени

4.3 Архитектура взаимодействия узлов системы

Для реализации алгоритма управления движением группы мобильных роботов в замкнутой среде была использована программная архитектура ROS. Она предоставляет мощный инструментарий для построения распределённых систем управления, в которых отдельные компоненты, называемые узлами, обмениваются информацией через специальные каналы — топики. Коммуникация между узлами осуществляется по принципу "издатель–подписчик": одни узлы публикуют сообщения в определённый топик, а другие — подписываются на эти сообщения и получают соответствующие данные.

Разработанный управляющий модуль функционирует в составе ROS-экосистемы и взаимодействует с рядом стандартных и собственных узлов. Центральным элементом системы выступает узел multi_tb3_planner, реализующий алгоритм построения траектории и согласования движения между роботами. Этот узел получает данные с одометрии каждого робота, а также данные о карте окружающего пространства, публикуемые в топике */map*. На основании полученной информации он рассчитывает вектор движения и публикует управляющие сигналы в топик */cmd_vel* соответствующего робота, задавая его линейную и угловую скорости.

Каждый робот в симуляции имеет свой собственный набор топиков, с уникальным идентификатором. Это позволяет избежать конфликтов при передаче данных и обеспечивает масштабируемость решения при увеличении количества агентов. Узел также публикует траектории движения роботов в специальные топики типа */path*, благодаря чему в среде *RViz* можно визуально отслеживать путь, пройденный каждым участником группы. Дополнительно, в систему внедрён механизм визуализации целей и пройденных точек с помощью массива маркеров, что позволяет удобно отображать состояние всей группы в процессе выполнения задачи.

Архитектура ROS-графа, построенная в процессе разработки, представляет собой типичную структуру распределённого взаимодействия. Каждый робот отправляет свои данные (например, одометрию) и получает управляющие сигналы независимо, в то время как централизованный управляющий узел анализирует всю информацию и координирует поведение всей группы. Благодаря использованию ROS-топиков и единого формата сообщений достигается высокая степень модульности и совместимости между компонентами.

Такое решение обеспечивает не только надёжное управление группой роботов, но и возможность гибкого расширения системы. При добавлении нового робота требуется лишь продублировать соответствующие топики и подписки, не затрагивая логики существующего алгоритма. Кроме того, с помощью инструментов ROS, таких как *rqt_graph*, можно в реальном времени отслеживать структуру графа сообщений, записывать данные сессии для последующего анализа и выявлять возможные узкие места в работе системы.

Таким образом, предложенная архитектура ROS-проекта отличается гибкостью, масштабируемостью и удобством визуализации, что делает её оптимальным выбором для разработки и апробации алгоритмов коллективного управления мобильными роботами в условиях ограниченного пространства и наличия препятствий.

4.4 Алгоритм управления группой роботов

Алгоритм осуществляет координацию группы роботов в пространстве с препятствиями, минимизируя общую длину траектории и предотвращая коллизии.

Инициализация. На первом этапе работы происходит инициализация начальных и целевых точек для каждого робота из группы. Задаются параметры безопасности – минимальное допустимое расстояние между роботами, минимально допустимое расстояние до препятствий и границы рабочей области.

Движение к цели. На втором этапе каждый робот собирает данные с датчиков об окружающей среде и строит по ним вектор расстояний L до препятствий в направлениях, заданными углами. На основе этих данных генерируются возможные направления движения. Среди массива возможных направлений выбирается оптимальное которая минимизирует расстояние до целевой точки.

Разрешение коллизий и выбор лидера. При отсутствии возможных направлений активируется процедура выбора лидера. Анализируя соседей в своей области видимости, в качестве лидера принимает ближайший робот, чья траектория к цели не заблокирована. Текущий робот корректирует свой вектор движения, следуя за лидером и сохраняя безопасную дистанцию. Для предотвращения заторов в узких зонах используется заданные значения приоритетов: объект запрашивает право прохода, и, если его движение возможно только при блокировке других роботов, движение регулируется в порядке приоритетов заданных изначально. Элемент системы с наибольшим приоритетом первым производит движение, остальные ожидают разрешения, которое будет получено после освобождения объектом узкого коридора.

Контроль достижения цели. На каждом шаге робот проверяет достижение цели, вычисляя расстояние d до конечной точки. Если d становится меньше порогового значения (например, 0,1 м), робот останавливается. Это означает

достижения целевой точки. Алгоритм работает итеративно и обновляет состояние на основе данных с датчиков в каждой точке траектории.

5. Рабочее проектирование системы

5.1 Создание карты для симуляции

В реализованном алгоритме используется бинарное представление окружающей среды, заданное через переменные среды. Эти переменные определяют размеры карты и препятствий на ней. Каждому пикселю соответствует состояние местности: темные пиксели считаются занятыми (препятствия), а белые пиксели обозначают свободное пространство, доступное для перемещения роботов

Для комплексного тестирования было создано три различных варианта карты с разными начальными условиями, имитирующими реальные складские помещения. Первая карта (рисунок 13) представляет собой замкнутое помещение с узкими складскими рядами, которые не позволяют двум роботам разъехаться одновременно, что создает высокую сложность для координации и планирования движения.

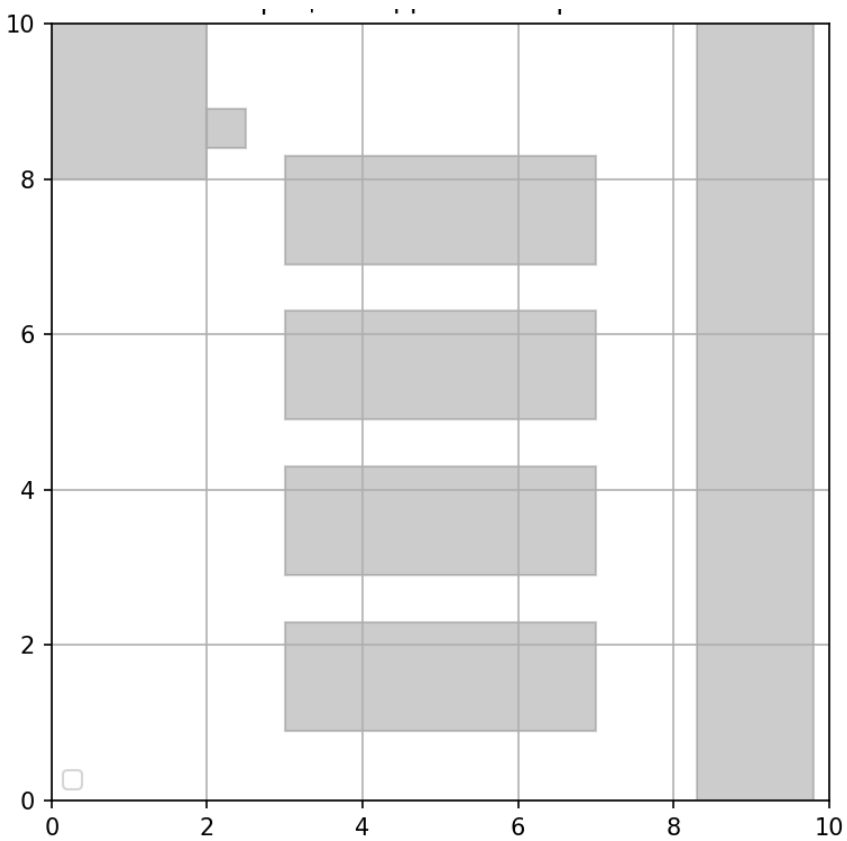


Рисунок 13 – Первый вариант карты

Вторая карта (рисунок 14) несколько шире, позволяя роботам разъезжаться с минимальным запасом пространства, что сохраняет необходимость точной координации траекторий и избегания столкновений.

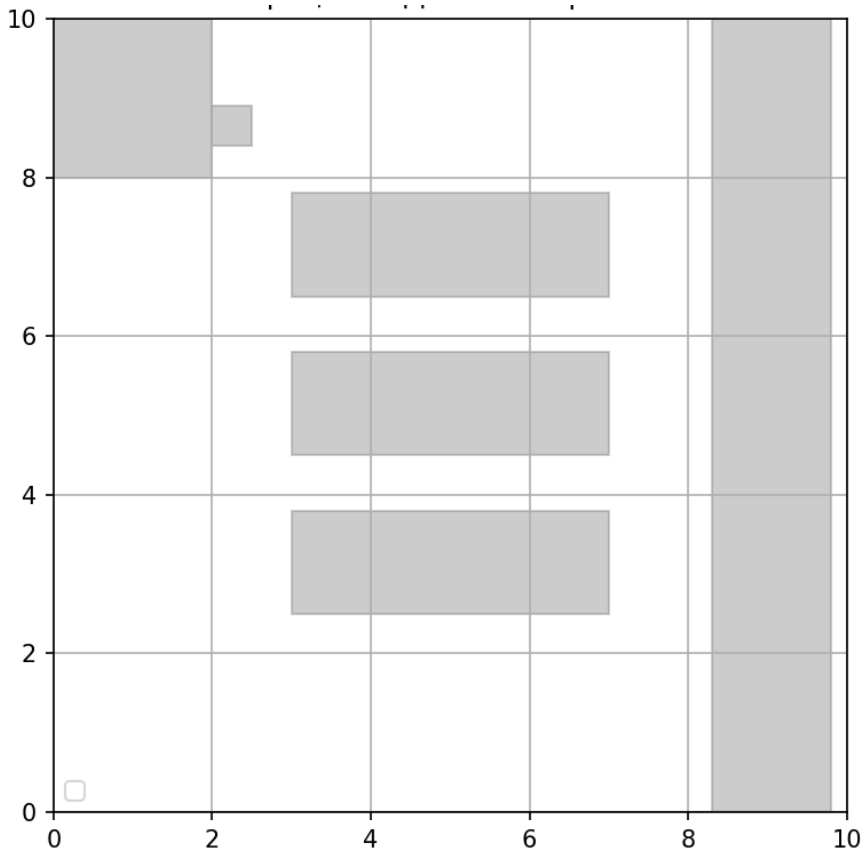


Рисунок 14 – Второй вариант карты

Третья карта (рисунок 15) имеет наиболее широкие проходы, облегчая движение роботов, но при этом содержащие повторяющиеся элементы (паттерны) и препятствия, усложняющие задачу точной локализации и навигации.

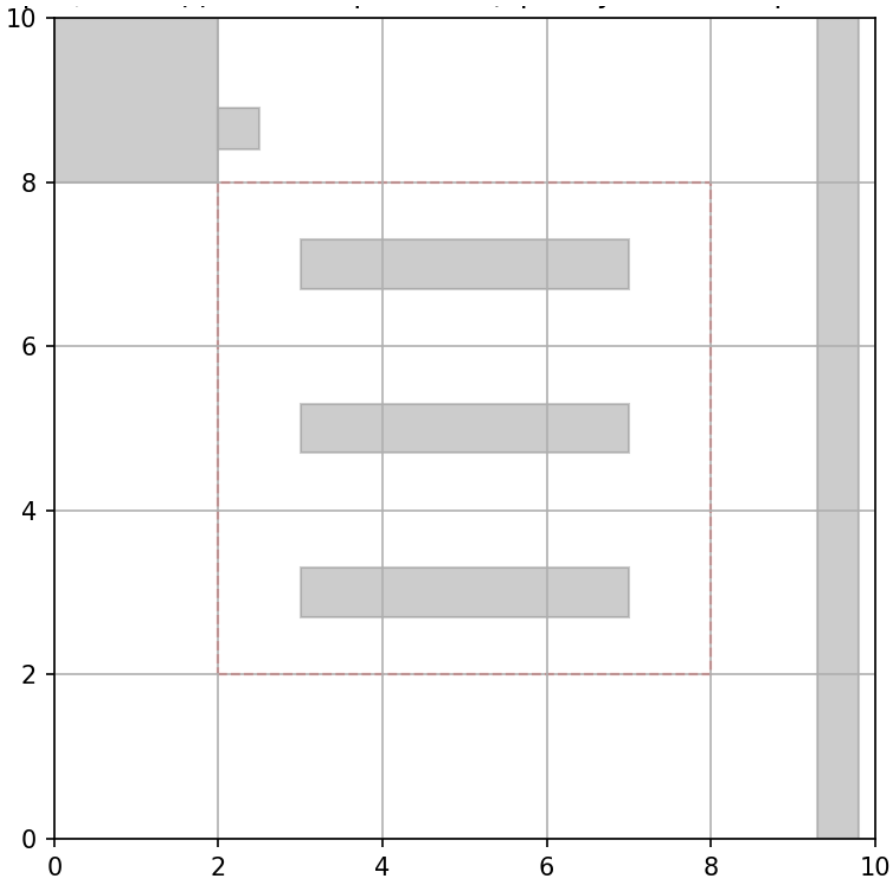


Рисунок 15 – Третий вариант карты

Карта, использованная для тестирования работы алгоритма, была выбрана исходя из нескольких важных критериев. Максимальная приближенность к реальному складскому помещению. Представляет собой замкнутую среду, содержащую узкие проходы и препятствия, характерные для типичного склада. Местность содержит повторяющиеся элементы (паттерны), которые специально подобраны для повышения сложности задачи локализации роботов и проверки точности работы алгоритма избегания препятствий и координации движения нескольких роботов.

Для проверки алгоритма движения роботов была задана начальная конфигурация с четырьмя роботами и круглыми препятствиями, размещёнными таким образом, чтобы сформировать сложные проходы и узкие пространства, в которых роботы вынуждены согласовывать свои траектории, избегая столкновений друг с другом и с препятствиями. Реализация предусматривает использование виртуальных датчиков, позволяющих каждому роботу локально оценивать обстановку и выбирать подходящую траекторию с учётом

безопасного расстояния до объектов среды и других роботов

5.2 Разработка модуля управления

Разработка алгоритма координации движения нескольких роботов выполнена на языке Python с использованием популярных научно-технических библиотек NumPy и Matplotlib, что обеспечивает одновременно лаконичность кода и высокую скорость прототипирования. Python, будучи высокоуровневым языком общего назначения с динамической строгой типизацией и автоматическим управлением памятью, позволяет инженеру-исследователю сосредоточиться на логике алгоритма, а не на низкоуровневых деталях распределения ресурсов.

В качестве коммуникационной основы используется библиотека rospy, предоставляющая полноценный интерфейс к ROS из-под Python. На базе этого инструментария был создан единый ROS-узел-планировщик, который в реальном времени собирает одометрические данные от четырёх роботов, анализирует текущее топологическое расположение и генерирует согласованные управляющие воздействия.

Алгоритм реализует итеративную схему движения в условиях замкнутой среды, используя карту занятости (/map). Библиотека NumPy применяется для высокоэффективных векторных операций над матрицей карты, вычисления расстояний между агентами и оценки свободного пространства.

На этапе инициализации начальных условий задаются целевые точки и стартовые параметры. Ограничения из функции (8 и 9) задаются через переменные MIN_DIST_BETWEEN и MIN_DIST_TO_OBST. Программный код приведен в листинге 1:

```
MIN_DIST_BETWEEN= 0.6
MIN_DIST_TO_OBST= 0.2
SENSOR_ANGLES= list(range(0, 360, 5))
NUM_ROBOTS = 4
```

Продолжение листинга 1

```
    start_positions = [
        np.array([5.0, 9.0]),
        np.array([1.0, 5.0]),
        np.array([5.0, 1.0]),
        np.array([8.0, 4.0]),
    ]
    goal_positions = [
        np.array([5.0, 1.0]),
        np.array([8.0, 4.0]),
        np.array([5.0, 9.0]),
        np.array([1.0, 5.0]),
    ]
    OBSTACLES = [
        (5.0, 3.0, 1.1),
        (5.0, 5.8, 1.1),]
```

Листинг 1 – Инициализация начальных условий

На втором этапе программы происходит обработка карты и проверка возникающих коллизий. Функция cb_map конвертирует данные, получаемые из узла /map в координаты. Is_occupied() проверяет наличие занятых ячеек в безопасном радиусе вокруг выбранной точки. Тем самым она обеспечивает выполнение пространственного ограничения (8 и 9)

```
    def cb_map(msg: OccupancyGrid):
    global map_info, map_grid
    map_info = msg.info
    data = np.array(msg.data,
dtype=np.int8).reshape(msg.info.height,

msg.info.width)
    map_grid = data.copy()
```

Продолжение листинга 2

```
    def is_occupied(x, y, safety):
    if map_grid is None:
        return False
    cell_rad = int(safety / map_info.resolution)
    ci, cj   = world_to_cell(x, y)
    i0 = max(ci - cell_rad, 0);      i1 = min(ci +
cell_rad, map_info.width-1)
    j0 = max(cj - cell_rad, 0);      j1 = min(cj +
cell_rad, map_info.height-1)
    patch = map_grid[j0:j1+1, i0:i1+1]
    return np.any(patch > 50)
```

Листинг 2 – Обработка карты и разрешение коллизий

Третьим этапом в алгоритме движения роботов стало создание функции find_next_position(), которая по заданному набору углов определяет следующую позицию робота. Сначала функция проверяет каждую потенциальную точку на наличие коллизий с препятствиями. После этого алгоритм исключает позиции, находящиеся слишком близко к другим роботам. Координаты, удовлетворившие всем условиям, помещаются в массив, из которого выбирается точка, минимизирующая евклидово расстояние до цели – аналог критерия (16). Вектор до этой точки формирует вектор движения для робота (13). В противном случае выбирается та координата, которая даёт минимальное расстояние до целевой точки робота.

Реализация данной функции приведена ниже:

```
def find_next_position(r_idx, robots, goal):
    cur = robots[r_idx]
    to_goal = goal - cur
    angles = SENSOR_ANGLES +
[math.degrees(math.atan2(to_goal[1], to_goal[0]))]
```

Продолжение листинга 3

```
      cands = []
    for deg in angles:
        ang = math.radians(deg)
        cand = cur + STEP_SIZE*np.array([math.cos(ang),
math.sin(ang)])

        if not (0 <= cand[0] <= WORLD_SIZE and 0 <=
cand[1] <= WORLD_SIZE):
            continue
        if is_occupied(cand[0], cand[1],
MIN_DIST_TO_OBST):
            continue

        if any(distance(cand, r)<MIN_DIST_BETWEEN and
r_idx>j
               for j,r in enumerate(robots) if
j!=r_idx):
            continue
        cands.append(cand)
    return None if not cands else min(cands, key=lambda
p: distance(p, goal))
```

Листинг 3 – Функция поиска следующей точки

При отсутствии у робота возможности прямого перемещения к цели, происходит процедура выбора лидера, которая помогает роботу, не имеющему прямого пути к цели, следовать за траекторией ближайшего соседа. Функция анализирует позиции всех роботов в окружении и ищет среди них ближайшего, но только при условии, что он не нарушает минимальной допустимой дистанции между роботами. Подобный подход позволяет ведомому роботу следовать за лидером, тем самым выходя из ситуаций, когда движение по

намеченной траектории невозможно. Важно отметить, что такая схема не приводит к коллизиям между ведомым и лидером: во-первых, выбор кандидата производится с учетом минимальной дистанции, а во-вторых, если ведомый в какой-то момент окажется слишком близко к своему лидеру (меньше порога MIN_DIST_BETWEEN), то в этом же шаге или на следующем он снова попытается выбрать путь по обычной процедуре find_next_position.

В листинге 4 приведён программный код реализации функции:

```
    def find_leader(r_idx, robots):
    me = robots[r_idx]
    others = sorted((distance(me, p), j) for j,p in
enumerate(robots) if j!=r_idx)
    return robots[others[0][1]] if others else None
```

Листинг 4 – Функция поиска лидера

6. Апробирование

Для наглядной демонстрации работы разработанного алгоритма планирования движения группы мобильных роботов в замкнутой среде с препятствиями был использован симулятор робототехники в трёхмерном пространстве — Gazebo, интегрированный с платформой ROS. Это программное обеспечение позволяет воспроизводить физически реалистичное поведение роботов и точно моделировать поведение в условиях, приближенных к реальному производственному складу. Выбор именно этой среды обусловлен широкими возможностями настройки параметров модели, высокой точностью расчётов и удобной интеграцией с ранее разработанными ROS-узлами управления.

В рамках апробации были проведены тестовые испытания алгоритма на трёх различных конфигурациях карт, имитирующих складские помещения различной плотности, ширины проходов и размещения препятствий. Эти карты содержат как широкие зоны для свободного маневрирования, так и узкие коридоры, создающие потенциальные ситуации блокировки движения. Таким образом, проверялась способность алгоритма адаптироваться к различным условиям среды.

Для каждого варианта карты были проведены тесты с разным числом роботов — 4 и 6. При этом исходные стартовые и целевые координаты для каждого робота задавались таким образом, чтобы минимизировать возможность их прямолинейного движения, что увеличивало сложность задачи планирования.

С целью объективной оценки эффективности работы алгоритма использовались три основных критерия, описанных ранее в главе 2.9:

- Средняя длина траектории одного робота — характеризует эффективность маршрута;
- Средний угол поворота — отражает плавность движения, важную для износостойкости и устойчивости при транспортировке;

- Отсутствие коллизий — является обязательным условием корректной работы алгоритма и определяет его пригодность для практического использования.

По итогам испытаний была составлена таблица 1, содержащая результаты вычисления целевых показателей, где $L_{\text{ср}}$ – средняя длина траектории одного робота из формулы (16), а θ — среднее значение угла поворота, характеризующее плавность траектории (согласно формулам (16-18).

Таблица 1 – результаты апробации

Номер теста	Конфигурация карты	Число роботов	$L_{\text{ср}}$, м	θ^{-},°
1	1	4	14,2	25,4
2	1	6	14,5	26,9
3	2	4	12,6	22,7
4	2	6	13,0	24,3
5	3	4	11,1	16,2
6	3	6	11,8	17,4

На рисунках 16-18 представлено тестирование разработанного алгоритма управления движением группы роботов в ограниченном пространстве с наличием препятствий и узкого коридора. В исходных параметрах алгоритма задано количество роботов равное 4 и стартовые и целевые позиции.

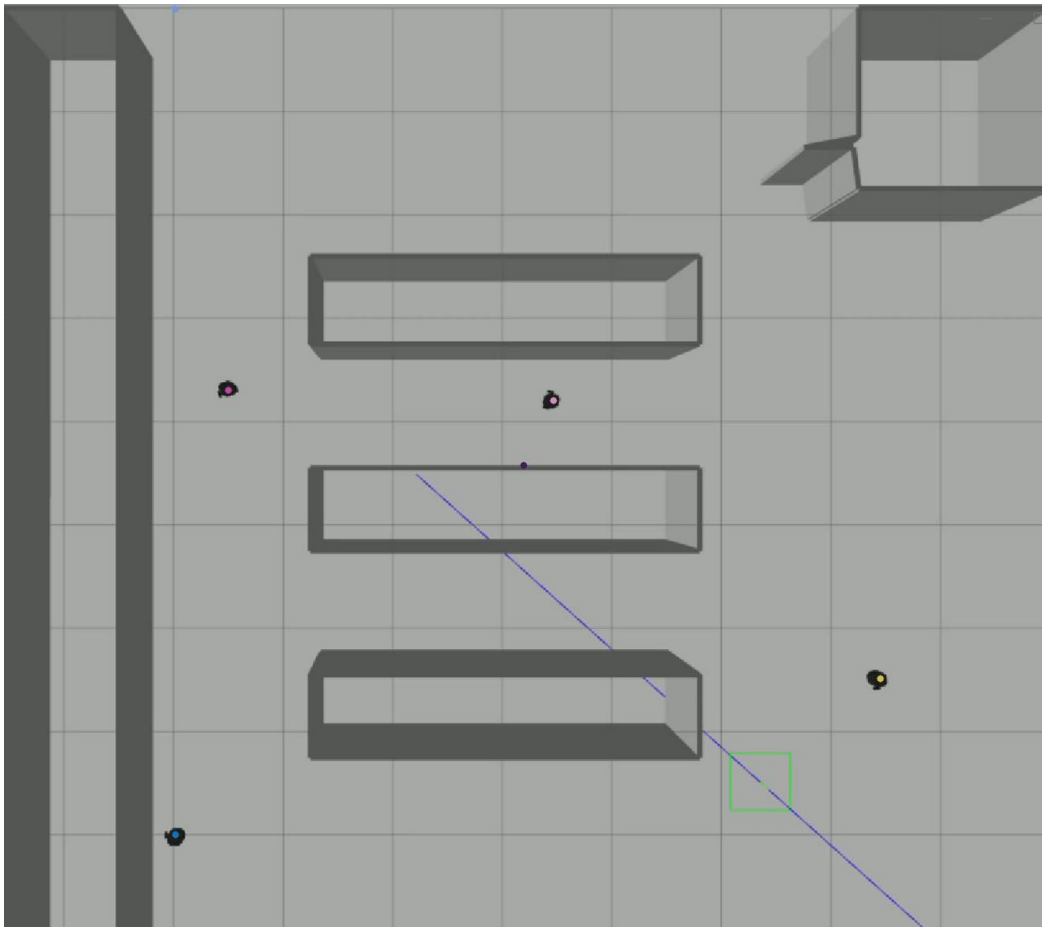


Рисунок 16 – Начальное положение

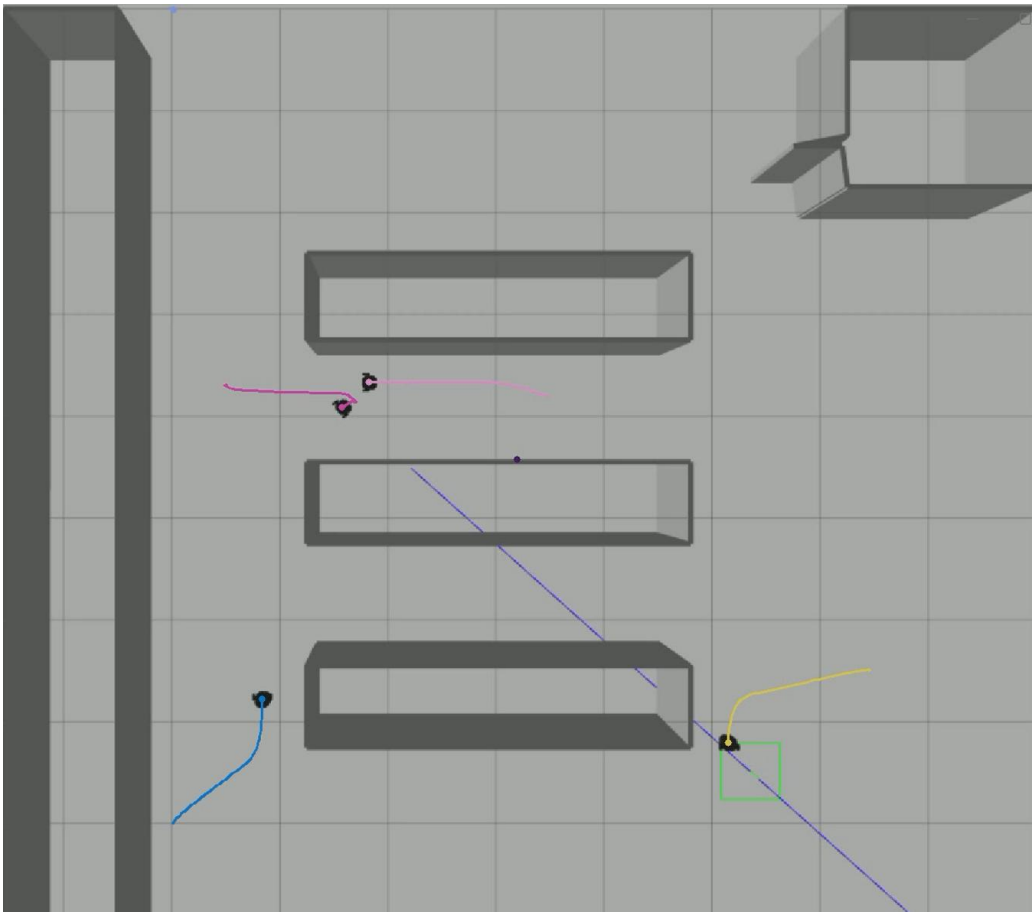


Рисунок 17 – Промежуточное положение

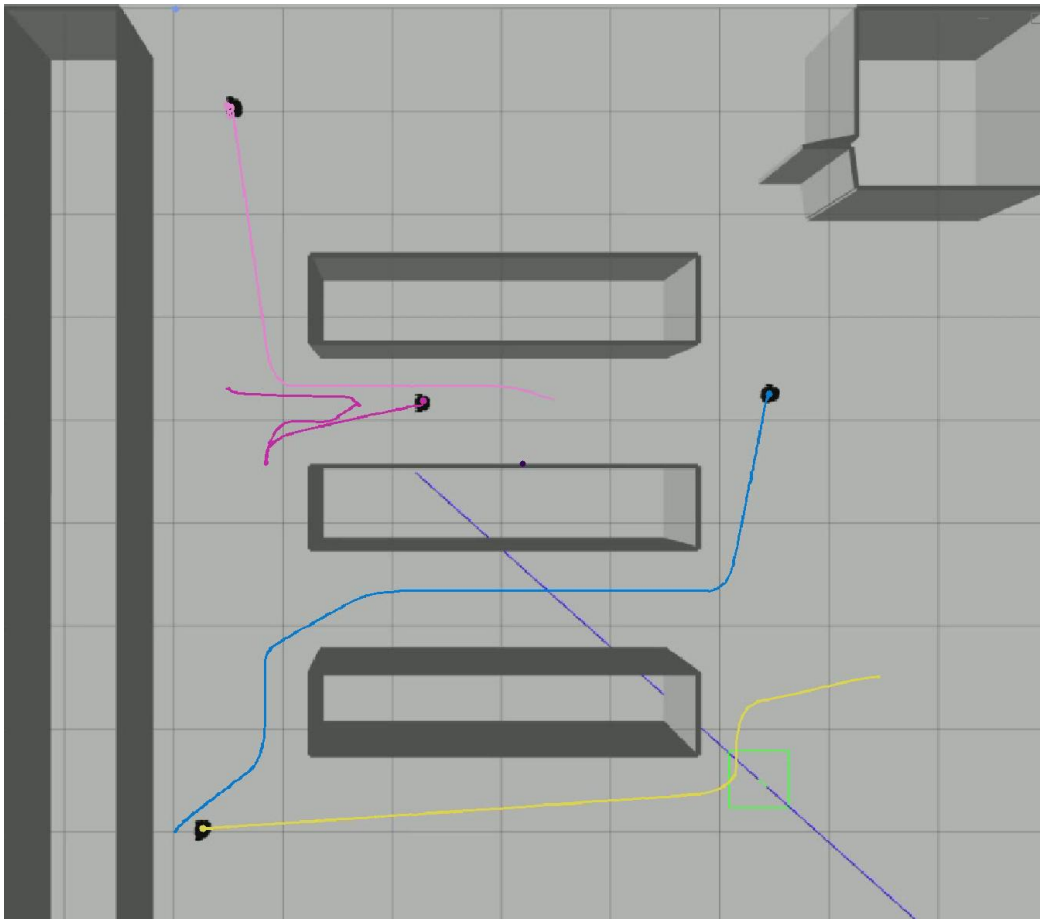


Рисунок 18 – Конечное положение

Каждый запуск проводился в условиях включённого контроля столкновений, имитации сканера лидаров, задержек сенсорных данных и одометрических шумов, что позволило максимально приблизить поведение системы к условиям реальной эксплуатации.

Результаты тестов подтверждают, что реализованный алгоритм:

- Эффективно справляется с координацией роботов в ограниченном пространстве;
- Демонстрирует стабильную работу при увеличении количества агентов;
- Гарантирует безопасность движения, соблюдая минимальные допустимые дистанции между объектами.

Таким образом, все шесть проведённых тестов завершились успешно: каждый из роботов достиг своей целевой позиции, не нарушив

пространственные ограничения, не допустив коллизий, при этом обеспечив высокие значения длины пути и плавности движения. Это подтверждает работоспособность и адаптивность предложенного подхода для решения задач навигации в складских помещениях, а также его потенциал для масштабируемого применения в промышленных условиях.

ЗАКЛЮЧЕНИЕ

Разработанная система обеспечивает эффективное планирование траекторий движения группы мобильных роботов в ограниченном пространстве с наличием разнообразных препятствий. В рамках выпускной квалификационной работы было детально проанализировано существующие решение задачи управления коллективным поведением мобильных роботов, после чего сформировано собственное техническое решение задачи оптимизации прокладки путей.

Ключевой особенностью предложенного подхода является разделение процесса построения траектории на два последовательных этапа. Такой поэтапный принцип работы даёт существенные преимущества: во-первых, облегчает тонкую настройку каждого модуля системы управления, во-вторых, значительно снижает общую вычислительную нагрузку, что особенно важно при ограниченных ресурсах аппаратной платформы.

Первый этап разработки включает в себя построение предварительной карты перемещений группы, анализ потенциальных зон конфликтов и формирование базового набора маршрутов с учётом геометрии среды и динамических свойств роботов. Второй этап фокусируется на уточнении и адаптации полученного маршрута под реальные условия: учитываются неточности измерений сенсоров, изменяющиеся параметры препятствий и необходимость оперативной коррекции курса в режиме реального времени. Благодаря наличию обратной связи система способна подстраиваться под неожиданные изменения, минимизируя риск столкновений и простоев.

В ходе выполнения курсового проекта были выполнены следующие задачи:

1. Проанализирован алгоритм управления группой мобильных роботов;
2. Рассмотрены актуальные варианты реализации системы управления;

3. Разработано техническое решение задачи оптимизации;
4. Реализована программа на языке Python;
5. Проведено тестирование работы системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Антонов В. О., Гурчинский М. М., Петренко В. И., Тебуева Ф. Б. Метод планирования траектории движения точки в пространстве с препятствием на основе итеративной кусочно-линейной аппроксимации // Системы управления, связи и безопасности. 2018. № 1. С. 168-182.

2. Афанасьев И. М., Сагитов А. Г., Данилов И. Ю., Магид Е. А. Навигация гетерогенной группы роботов (БПЛА и БНР) через лабиринт в 3D симуляторе Gazebo методом вероятностной дорожной карты // Второй Всероссийский научно-практический семинар «Беспилотные транспортные средства с элементами искусственного интеллекта» Труды семинара. – Санкт-Петербург Политехника-сервис, 2015. – С. 18-25.

3. Бройнль Т. Встраиваемые робототехнические системы: проектирование и применение мобильных роботов со встроенными системами управления. Ижевск: Ижевский институт компьютерных исследований, 2012. 520 с.

4. Васильев С. Н., Браништов С. А., Бузиков М. Э., Морозов Н. Ю. Групповое управление автономными мобильными аппаратами // Материалы 10-й Всероссийской мультиконференции по проблемам управления (МКПУ2017). – Дивноморское, 2017. – С. 251-252.

5. Гайдук А. Р., Капустян С. Г., Шаповалов И. О. Алгоритм управления движением группы мобильных роботов в условиях неопределенности // Инженерный вестник Дона. 2018. № 3 (50). С. 1-13.

6. Градецкий В. Г., Ермолов И. Л., Князьков М. М., Семёнов Е. А., Собольников С. А., Суханов А. Н. Вопросы взаимодействия роботов в группе при выполнении единой транспортной задачи // Материалы 10-й Всероссийской мультиконференции по проблемам управления (МКПУ-2017). – Дивноморское, 2017. – С. 265-267.

7. Даринцев О. В., Мигранов А. Б. Двухуровневая интеллектуальная система планирования движений мобильных роботов // Труды Института

механики им. Р.Р. Мавлютова Уфимского научного центра РАН. 2014. Т. 10. С. 50-54. DOI: 10.21662/uim2014.1.009

8. Павлов А. С., Рябцев С. С., Ахмедов С. Р., Трофимюк О. И. Разработка алгоритма децентрализованного управления группой беспилотных автомобилей на основе метода роя частиц // Труды VI Всероссийской научной конференции «Информационные технологии интеллектуальной поддержки принятия решений». – Уфа, 2018. – С. 168-173.

9. Петренко В. И., Тебуева Ф. Б., Павлов А. С., Антонов В. О. Метод планирования пути при формировании конфигурации многофункционального модульного робота с использованием роевой стратегии управления // Труды VII Всероссийской научной конференции «Информационные технологии интеллектуальной поддержки принятия решений». – Уфа, 2019. – С. 184-189.

10. Пшихопов В. Х., Медведев М. Ю. Планирование движения группы подвижных объектов в двумерной среде с препятствиями // Известия Южного федерального университета. Технические науки. 2016. №2 (175). С. 6-22.

11. Халимов Н. Р., Мефедов А. В. Распределенная сетецентрическая система управления группой ударных беспилотных летательных аппаратов // Системы управления, связи и безопасности. 2019. №3. С. 1-13. DOI:10.24411/2410-9916-2019-10301

12. Юдинцев Б. С. Синтез нейросетевой системы планирования траекторий для группы мобильных роботов // Системы управления, связи и безопасности. 2019. № 4. С. 163-186. DOI: 10.24411/2410-9916-2019-10406

13. Ющенко А. С., Михайлов Б. Б., Назарова А. В. Автономные мобильные роботы - навигация и управление // Известия Южного федерального университета. 2016. № 2 (175). С. 48-65.

14. Kennedy J., Eberhart R. Particle swarm optimization // Proceedings of ICNN'95 - International Conference on Neural Networks. 1995. Vol. 4. P. 19421948. DOI: 10.1109/ICNN.1995.488968. 13. Björnsson Y., Enzenberger M., Holte R., Schaeffer J. Fringe Search: Beating A* at Pathfinding on Game Maps // Proceedings

of the 2005 IEEE Symposium on Computational Intelligence and Games (CIG05). – Colchester, Essex, UK, 2005. – P. 125-132.

15. Wahab M. N. A., Lee C. M., Akbar M. F., Hassan F. H. Path Planning for Mobile Robot Navigation in Unknown Indoor Environments Using Hybrid PSOFS Algorithm // IEEE Access. 2020. Vol. 8. P. 161805-161815. DOI:10.1109/ACCESS.2020.3021605

16. Zhang, H.-Y.; Lin, W.-M.; Chen, A.-X. Path Planning for the Mobile Robot: A Review // Symmetry, 2018, Vol. 10, No. 10, Art. 434, 17 pp.; submitted 12 Jul. 2018; published 04 Oct. 2018; updated 15 Nov. 2018. — DOI: 10.3390/sym10100450 — ISSN 2073-8994; EISSN 2073-8994; URL: https://www.mdpi.com/2073-8994/10/10/450

17. Kumar, P.; Singh, R.; Sharma, V. Distributed Multi-Robot Path Planning: A Comparative Study of Centralized and Decentralized Approaches // Robotics and Autonomous Systems, 2021, Vol. 138, Art. 101419, 14 pp.; received 22 Dec. 2020; revised 10 Mar. 2021; accepted 03 Apr. 2021. — DOI: 10.1016/j.robot.2021.103981 — ISSN 0921-8890; EISSN 1872-8091; Publisher: Elsevier.

18. Li, X.; Wu, Y.; Xie, L. Real-Time Obstacle Avoidance in Multi-Robot Systems Using Enhanced Potential Field Method // *Journal of Intelligent & Robotic Systems*, 2019, Vol. 95, Nos. 3–4, pp. 563–577; first online 15 Jan. 2019; published in print 07 Mar. 2019. — DOI: 10.1007/s10846-019-00931-2 — ISSN 0921-0296 (Print); ISSN 1573-0409 (Online); Springer.

19. Björnsson Y., Enzenberger M., Holte R., Schaeffer J. Fringe Search: Beating A* at Pathfinding on Game Maps // Proceedings of the 2005 IEEE Symposium on Computational Intelligence and Games (CIG'05). — Colchester, Essex, UK, 22–24 2005. — P. 125–132. — ISBN 0-7803-9212-5.

20. Chen, Y.; Huang, Z.; Zhao, W.; Xu, G. Real-Time Trajectory Optimization for Mobile Robots via Nonlinear Model Predictive Control // *IEEE Transactions on Robotics*, 2021, Vol. 37, No. 5, pp. 1349–1363; received 15 Oct. 2020; revised 02 Feb. 2021; accepted 18 Mar. 2021; published 05 Apr. 2021. — DOI:

10.1109/TRO.2021.3067890 — ISSN 1552-3098 (Print); EISSN 1941-0468 (Online); IEEE.

ПРИЛОЖЕНИЕ А

Программный код системы

```
multi_robot_planner_rviz.py
ROS Noetic + Gazebo + RViz  (Ubuntu 20.04)
import rospy, math, numpy as np
from nav_msgs.msg          import OccupancyGrid, Path,
Odometry
from geometry_msgs.msg     import PoseStamped,
TransformStamped, Twist
from visualization_msgs.msg import Marker, MarkerArray
import tf2_ros

NUM_ROBOTS       = 4
WORLD_SIZE       = 10.0
MIN_DIST_BETWEEN = 0.6
MIN_DIST_TO_OBST = 0.2
STEP_SIZE        = 0.3
SENSOR_ANGLES    = list(range(0, 360, 5))
MAX_ITERS        = 500
GOAL_THRESHOLD   = 0.3
STUCK_THRESHOLD  = 5
MIN_MOVEMENT     = STEP_SIZE * 0.1

Kp_ang = 1.5
Kp_lin = 0.5

start_positions = [
    np.array([4.5, 6.2]),
    np.array([7.5, 6.3]),
```

```
    np.array([8.0, 2.0]),
    np.array([2.5, 4.5]),
]
goal_positions = [
    np.array([7.5, 9.3]),
    np.array([5.5, 6.2]),
    np.array([2.5, 6.5]),
    np.array([8.0, 2.0]),
]
map_info = None
map_data = None
def map_cb(msg: OccupancyGrid):
    global map_info, map_data
    map_info = msg.info
    map_data = msg.data

odom_pos = [None]*NUM_ROBOTS    # [np.array([x,y]), ...]
odom_yaw = [None]*NUM_ROBOTS    # [theta, ...]
def make_odom_cb(idx):
    def odom_cb(msg: Odometry):
        x = msg.pose.pose.position.x
        y = msg.pose.pose.position.y
        # извлекаем yaw из кватерниона
        q = msg.pose.pose.orientation
        siny = 2.0*(q.w*q.z + q.x*q.y)
        cosy = 1.0 - 2.0*(q.y*q.y + q.z*q.z)
        theta = math.atan2(siny, cosy)
        odom_pos[idx] = np.array([x,y])
        odom_yaw[idx] = theta
    return odom_cb
```

```
def world_to_cell(x,y):
    i = int((x - map_info.origin.position.x) /
map_info.resolution)
    j = int((y - map_info.origin.position.y) /
map_info.resolution)
    i = min(max(i,0), map_info.width-1)
    j = min(max(j,0), map_info.height-1)
    return i,j

def is_collision_grid(pos, min_d):
    if map_data is None:
        return False
    x,y = pos
    ci,cj = world_to_cell(x,y)
    cell_rad = int(min_d / map_info.resolution)
    i0,i1 = max(ci-cell_rad,0), min(ci+cell_rad,
map_info.width-1)
    j0,j1 = max(cj-cell_rad,0), min(cj+cell_rad,
map_info.height-1)
    for jj in range(j0, j1+1):
        for ii in range(i0, i1+1):
            if map_data[jj*map_info.width + ii] > 50:
                return True
    return False
def distance(p1,p2):
    return np.hypot(*(p1-p2))

def find_next(idx, robots, goal):
    cur = robots[idx]
    vx,vy = goal - cur
```

```
    goal_ang = math.degrees(math.atan2(vy,vx))
    angles = SENSOR_ANGLES + [goal_ang]
    cands = []
    for deg in angles:
        rad = math.radians(deg)
        cand = cur +
STEP_SIZE*np.array([math.cos(rad),math.sin(rad)])
        if not (0<=cand[0]<=WORLD_SIZE and
0<=cand[1]<=WORLD_SIZE):
            continue
        if is_collision_grid(cand, MIN_DIST_TO_OBST):
            continue
        if any(distance(cand,r)<MIN_DIST_BETWEEN and
idx>j
               for j,r in enumerate(robots) if j!=idx):
            continue
        cands.append(cand)
    if not cands:
        return None
    return min(cands, key=lambda p: distance(p,goal))

def find_leader(idx, robots):
    me = robots[idx]
    others = sorted((distance(me,r),i) for i,r in
enumerate(robots) if i!=idx)
    return robots[others[0][1]] if others else None

def escape(idx, robots):
    cur = robots[idx]
    for _ in range(8):
```

```
        ang = np.random.uniform(0,2*math.pi)
        cand = cur +
STEP_SIZE*np.array([math.cos(ang),math.sin(ang)])
        if not (0<=cand[0]<=WORLD_SIZE and
0<=cand[1]<=WORLD_SIZE):
            continue
        if is_collision_grid(cand,MIN_DIST_TO_OBST):
            continue
        if any(distance(cand,r)<MIN_DIST_BETWEEN for j,r
in enumerate(robots) if j!=idx):
            continue
        return cand
    return None

stuck = [0]*NUM_ROBOTS
def plan_step(robots):
    global stuck
    nxt = []
    for i in range(NUM_ROBOTS):
        cur,goal = robots[i], goal_positions[i]
        if distance(cur,goal) <= GOAL_THRESHOLD:
            nxt.append(cur); stuck[i]=0; continue
        if stuck[i] > STUCK_THRESHOLD:
            esc = escape(i,robots)
            if esc is not None:
                nxt.append(esc); stuck[i]=0; continue
        cand = find_next(i,robots,goal)
        if cand is None:
            lead = find_leader(i,robots)
            if lead is not None and np.linalg.norm(lead-
```

```
cur)>0:
                cand = cur + STEP_SIZE*(lead-
cur)/np.linalg.norm(lead-cur)
        if cand is None: cand = cur
        stuck[i] = stuck[i]+1 if
distance(cur,cand)<MIN_MOVEMENT else 0
        nxt.append(cand)
    return nxt

def make_markers(robots):
    arr = MarkerArray()
    # цели — зелёные
    for k,g in enumerate(goal_positions):
        m = Marker(); m.header.frame_id="map";
m.header.stamp=rospy.Time.now()
        m.id = 100+k; m.type = Marker.SPHERE
        m.pose.position.x,m.pose.position.y = g
        m.scale.x = m.scale.y = m.scale.z = 0.25
        m.color.r,m.color.g,m.color.b,m.color.a = 0,1,0,1
        arr.markers.append(m)
    # роботы — цветные
    cols = [(0,0,1),(0,1,0),(1,0,0),(0.6,0,1)]
    for k,p in enumerate(robots):
        m = Marker(); m.header.frame_id="map";
m.header.stamp=rospy.Time.now()
        m.id = 200 + k; m.type = Marker.SPHERE
        m.pose.position.x,m.pose.position.y = p
        m.scale.x = m.scale.y = m.scale.z = 0.3
        r,g,b = cols[k]
        m.color.r,m.color.g,m.color.b,m.color.a = r,g,b,1
```

```
        arr.markers.append(m)
    return arr
def main():
    rospy.init_node("multi_robot_planner")

    # карта
    rospy.Subscriber("/map", OccupancyGrid, map_cb)

    # odom
    for i in range(NUM_ROBOTS):
        rospy.Subscriber(f"/tb3_{i}/odom", Odometry,
make_odom_cb(i))

    # cmd_vel для Gazebo
    cmd_pubs = [rospy.Publisher(f"/tb3_{i}/cmd_vel",
Twist, queue_size=1)
                for i in range(NUM_ROBOTS)]
    path_pubs = [rospy.Publisher(f"/tb3_{i}/path", Path,
queue_size=1)
                 for _ in range(NUM_ROBOTS)]
    path_msgs =
[Path(header=rospy.Header(frame_id="map"))
                 for _ in range(NUM_ROBOTS)]

    marker_pub =
rospy.Publisher("/visualization_marker_array",
                                MarkerArray,
queue_size=1, latch=True)

    tf_br = tf2_ros.TransformBroadcaster()
```

```
    rate = rospy.Rate(5)

    START_DELAY  = rospy.Duration(20.0)        # 10 с
    t0 = None

    it = 0
    while not rospy.is_shutdown() and it < MAX_ITERS:
        # ждём карту и одну ODOM для каждого
        if map_data is None or any(odom_pos[i] is None
for i in range(NUM_ROBOTS)):
            rate.sleep(); continue
        if t0 is None:
            t0 = rospy.Time.now()

        if rospy.Time.now() - t0 < START_DELAY:
            zero = Twist()
            for pub in cmd_pubs:
                pub.publish(zero)
            rate.sleep()
            continue
        robots = [odom_pos[i].copy() for i in
range(NUM_ROBOTS)]
        new_pos = plan_step(robots)

        for i in range(NUM_ROBOTS):
            cur = odom_pos[i]
            yaw = odom_yaw[i]
            tgt = new_pos[i]
            dx, dy = tgt - cur
```

```
            dist = math.hypot(dx, dy)
            target_ang = math.atan2(dy, dx)
            err_ang = math.atan2(math.sin(target_ang-
yaw),
                                 math.cos(target_ang-
yaw))

            tw = Twist()
            tw.angular.z = Kp_ang * err_ang
            tw.linear.x  = max(0.0, Kp_lin * dist *
math.cos(err_ang))
            cmd_pubs[i].publish(tw)

            # --- Path для RViz ---
            ps = PoseStamped()
            ps.header.frame_id = "map"
            ps.header.stamp = rospy.Time.now()
            ps.pose.position.x, ps.pose.position.y = cur
            ps.pose.orientation.w = 1.0
            path_msgs[i].poses.append(ps)
            path_pubs[i].publish(path_msgs[i])

            tfm = TransformStamped()
            tfm.header.stamp = rospy.Time.now()
            tfm.header.frame_id = "map"
            tfm.child_frame_id  = f"tb3_{i}"
            tfm.transform.translation.x = float(cur[0])
            tfm.transform.translation.y = float(cur[1])
            tfm.transform.rotation.w = 1.0
            tf_br.sendTransform(tfm)
```

```
        marker_pub.publish(make_markers(robots))

        it += 1
        rate.sleep()

    rospy.loginfo("Simulation finished")

if __name__=="__main__":
    try:
        main()
    except rospy.ROSInterruptException:
        pass
```

ПРИЛОЖЕНИЕ Б

Программный код запуска логики программы

```
<launch>

  <include file="$(find
multi_tb3_nav)/launch/multi_tb3_gazebo.launch"/>



  <node name="planner" pkg="multi_tb3_nav"
type="planner_node.py" output="screen"/>

</launch>
```

ПРИЛОЖЕНИЕ В

Графическая часть

РАЗРАБОТКА СИСТЕМЫ ПЛАНИРОВАНИЯ ТРАЕКТОРИИ ДВИЖЕНИЯ ГРУППЫ МОБИЛЬНЫХ РОБОТОВ В ЗАМКНУТОЙ СРЕДЕ С ПРЕПЯТСТВИЯМИ

Студент РК9-41М – Е.О. Ольхов

Научный руководитель - М.В. Овсянников

1

ЦЕЛЬ И ЗАДАЧИ

Назначение разрабатываемой системы – обеспечение автономности транспортировки грузов на складском помещении с помощью группы мобильных роботов

Поставлены следующие задачи:

- Составить техническое задание для разрабатываемой системы
- Проанализировать объект и требования
- Выбрать архитектуру управления и навигации
- Сравнить алгоритмы планирования
- Разработать алгоритмы основных функций программ
- Разработать исходный код программной части на языке python на базе открытой робототехнической платформы ros

2

КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ

Система управления: централизованная

В данной работе мной была рассмотрена централизованная система управления поскольку такой подход требует наличия только одного высокопроизводительного вычислительного центра, что значительно сокращает затраты на оснащение каждого отдельного робота сложной и дорогостоящей аппаратурой.

Навигация: Лидар

3

ВЫБОР БАЗОВОГО АЛГОРИТМА

В ходе анализа существующих методов в качестве базового для данной работы был выбран жадный алгоритм. Основная причина – его простота реализации и оперативная реакция на локальные изменения окружения, что особенно важно в частично неизвестной или динамически меняющейся среде.

Критерии выбора алгоритма:
- время расчёта
- длина пути
- гладкость
- масштабируемость

4

ПОСТАНОВКА ЗАДАЧИ

$$s_i(t) = f(s_i(t-1), u_i(t)) \quad , \text{T} = 0,1, \dots , \quad \text{K}. (1)$$

$$Y = f_i + f_{t+i} + \cdots + f_k \quad (2)$$

В качестве показателя эффективности примем длину траектории, необходимую для перемещения всех роботов группы из начального положения в целевое. Подставив выражения (1) в (2), получим аддитивную целевую функцию:

$Y = \sum_{t=1}^{k} f(s_i(t-1), u_i(t))$

5

ПОСТАНОВКА ЗАДАЧИ

- Таким образом, задачу пошаговой оптимизации процесса планирования траектории движения группы мобильных роботов в неизвестной замкнутой среде с препятствиями можно сформулировать так: определить для каждого робота группы $r_i \in R$ набор управлений $u_i(1)$, $u_i(2)$, … ,$u_i(k)$, переводящих систему из начального состояния $s_i(0)$, $i=\{1,...,N\}$ в конечное состояние $s_i(k)$, $i=\{1,...,N\}$ с учетом ограничений (7) так, чтобы целевая функция (8) достигла минимума:
  - $Y \to min$

6

АНАЛИЗ ТЕКУЩЕГО СОСТОЯНИЯ РОБОТА, ВЫБОР ПРОМЕЖУТОЧНОЙ ПОЗИЦИИ

Блок-схема процедуры выбора управления.

7

ПРОЦЕДУРА ВЫБОРА ЛИДЕРА

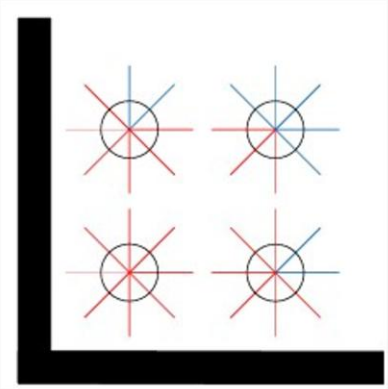


Иллюстрация проблемной ситуации

8

Блок-схема процедуры выбора лидера

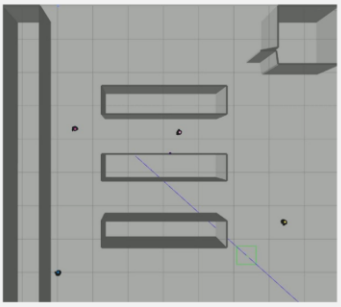
9

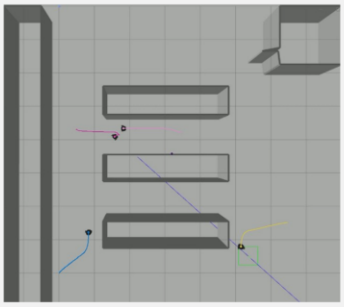
АЛГОРИТМ ДВИЖЕНИЯ ГРУППЫ МОБИЛЬНЫХ РОБОТОВ И КРИТЕРИИ ОЦЕНКИ

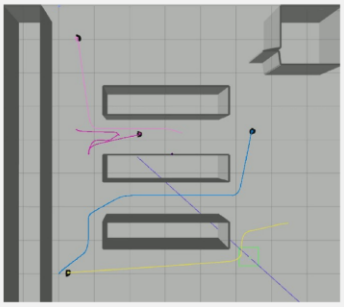
Блок-схема методики планирования траектории движения группы мобильных роботов в неизвестной замкнутой среде с препятствиями

10

АПРОБИРОВАНИЕ







11

РЕЗУЛЬТАТЫ АПРОБИРОВАНИЯ

Номер теста	Конфигурация карты	Число роботов	$L_{ср}$, м	θ^{-},°
1	1	4	14,2	25,4
2	1	6	14,5	26,9
3	2	4	12,6	22,7
4	2	6	13,0	24,3
5	3	4	11,1	16,2
6	3	6	11,8	17,4

12

ЗАКЛЮЧЕНИЕ

- Разработанная система обеспечивает эффективное планирование траекторий движения группы мобильных роботов в ограниченном пространстве с наличием разнообразных препятствий. В рамках выпускной квалификационной работы было детально проанализировано существующие решение задачи управления коллективным поведением мобильных роботов, после чего сформировано собственное техническое решение задачи оптимизации прокладки путей.

13

СПАСИБО ЗА ВНИМАНИЕ!

14